

Introduction to Artificial Neural Networks

Maciej Piasecki, Konrad Wojtasik & Arkadiusz Janz

Neurons

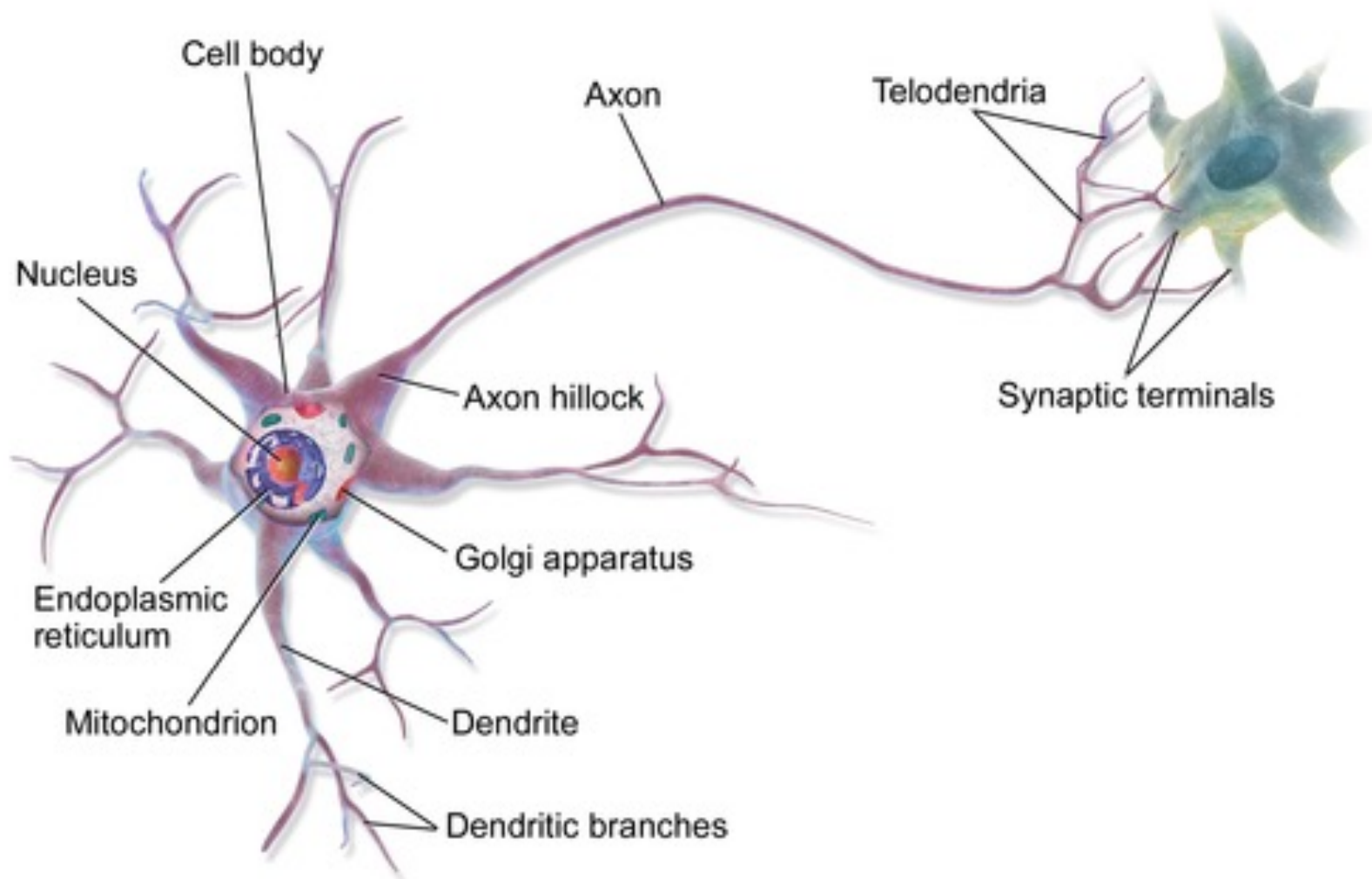
○ Biological neuron

- brain – neurons, but also *glial cells*,
- many inputs, one single output
- a cell collecting signals
- generating (triggering) an output signal, when the activation level reaches some threshold
- complicated electrical-chemical transmission
- non-linear characteristics
 - * slow building up activation $\Rightarrow$ sudden, dynamic changes around the threshold

○ Artificial neuron

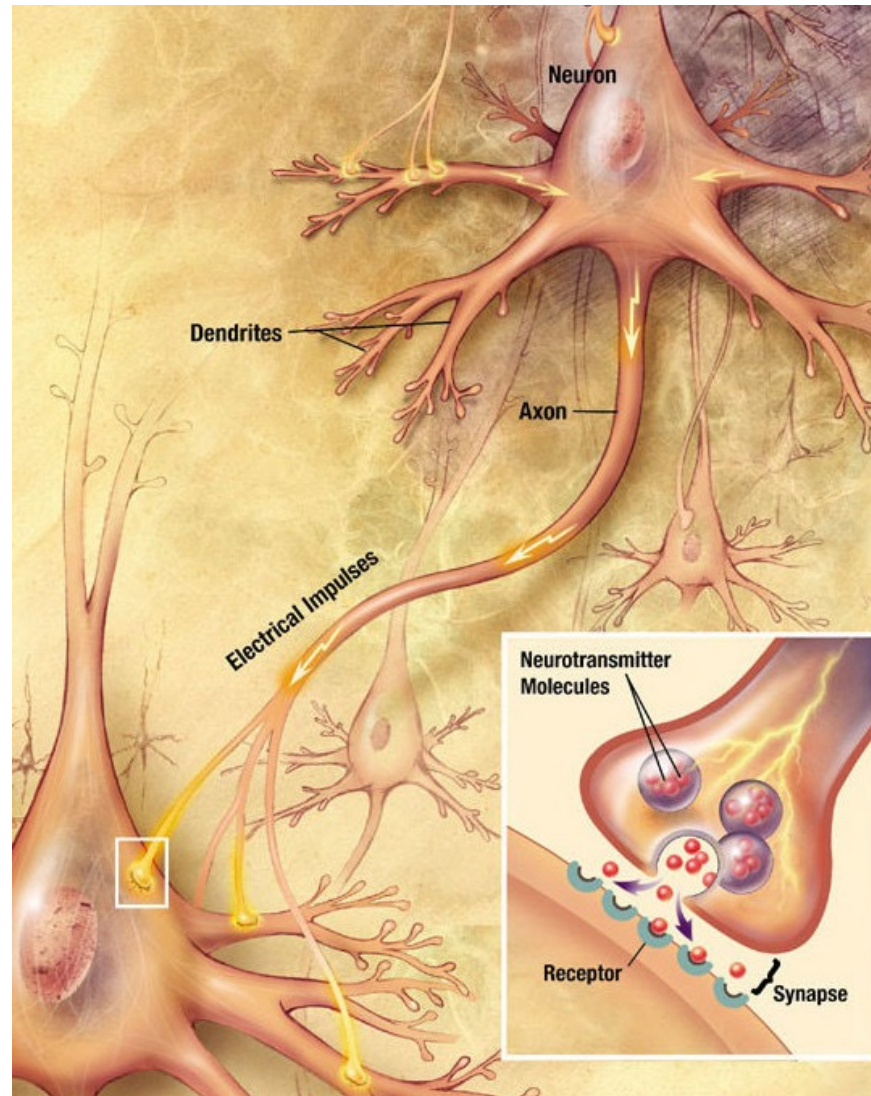
- a model — a function of many inputs and one output
- parameterised by many parameters
- first of all by **weights** on input signals

Natural neuron: structure



- o https://en.wikipedia.org/wiki/Neuron#/media/File:Blausen_0657_MultipolarNeuron.png

Natural neuron: signal transmission and processing



o https://en.wikipedia.org/wiki/File:Chemical_synapse_schema_cropped.jpg

Artificial neuron

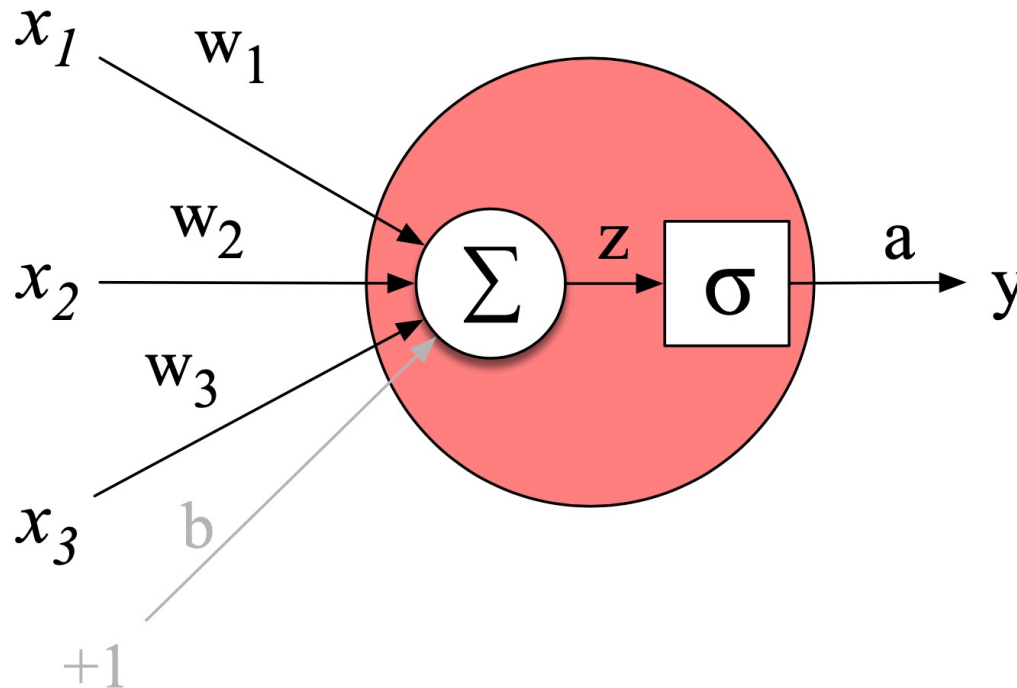
○ A function: $\mathcal{R}^n \rightarrow \mathcal{R}$

□ transfer function (activation function)

□ Σ - dot product: $\sum w_i x_i + b$

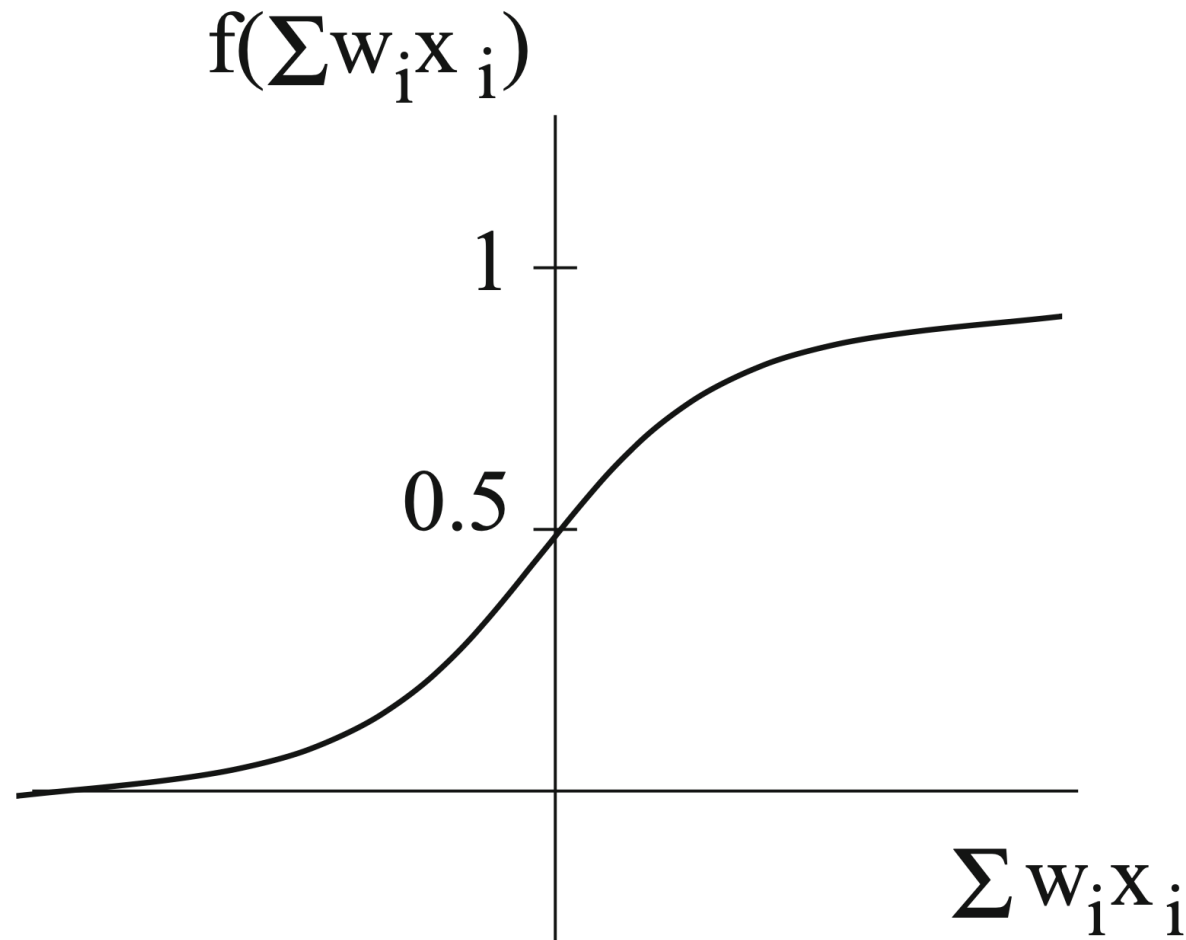
* where x_i — input signal, w_i — weight on the i -th input, b — bias

$$f(\Sigma) = \frac{1}{1 + e^{-\Sigma}}$$



(Juratsky & Martin, pp. 135, draft Jan 2022)

Activation function – sigmoid



(Kubot, pp. 92, 2017)

Other popular activation functions

$$\sigma(y) = \frac{1}{1 + \exp(-y)}$$

sigmoid (logistic),

$$\sigma(y) = \tanh(y) = \frac{\exp(y) - \exp(-y)}{\exp(y) + \exp(-y)}$$

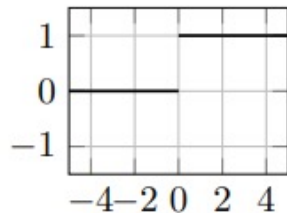
hyperbolic tangent,

$$\sigma(y) = \max\{y, 0\}$$

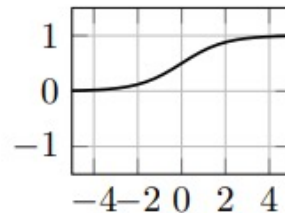
rectified linear unit (ReLU),

$$\sigma(y) = \max\{\alpha y, y\}$$

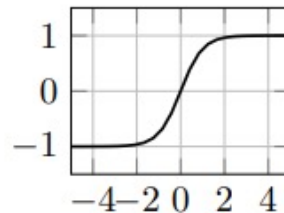
leaky ReLU.



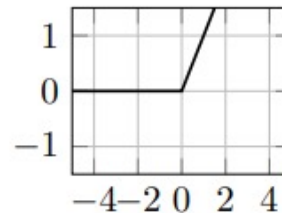
(a) Heaviside



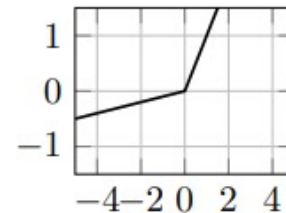
(b) sigmoid



(c) tanh



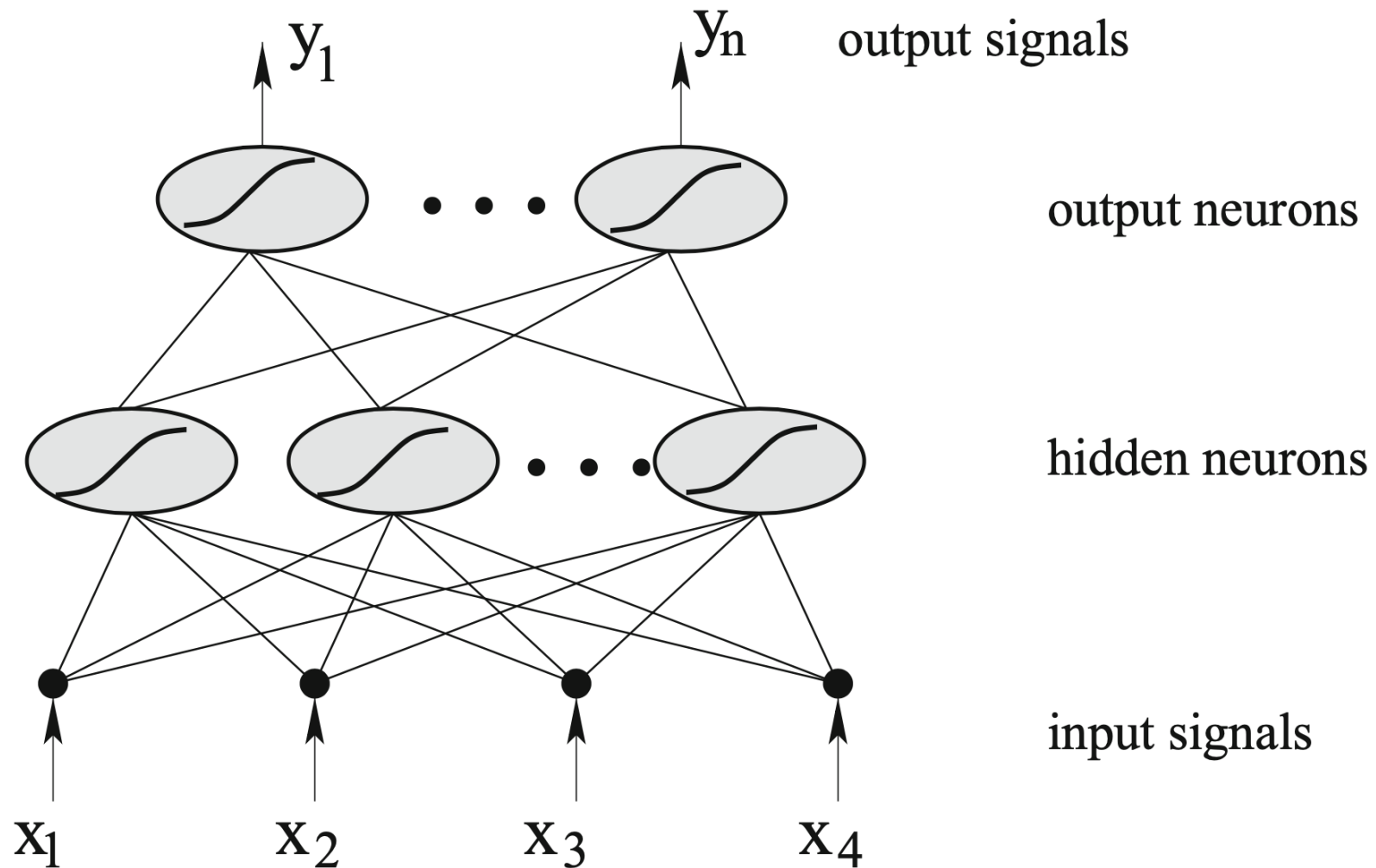
(d) ReLU



(e) leaky ReLU

Lecture Notes: Neural Network Architectures, **Evelyn Herberg**

Multilayer Perceptron Neural Network



- bias omitted (zeroed) (Kubot, pp. 92, 2017)

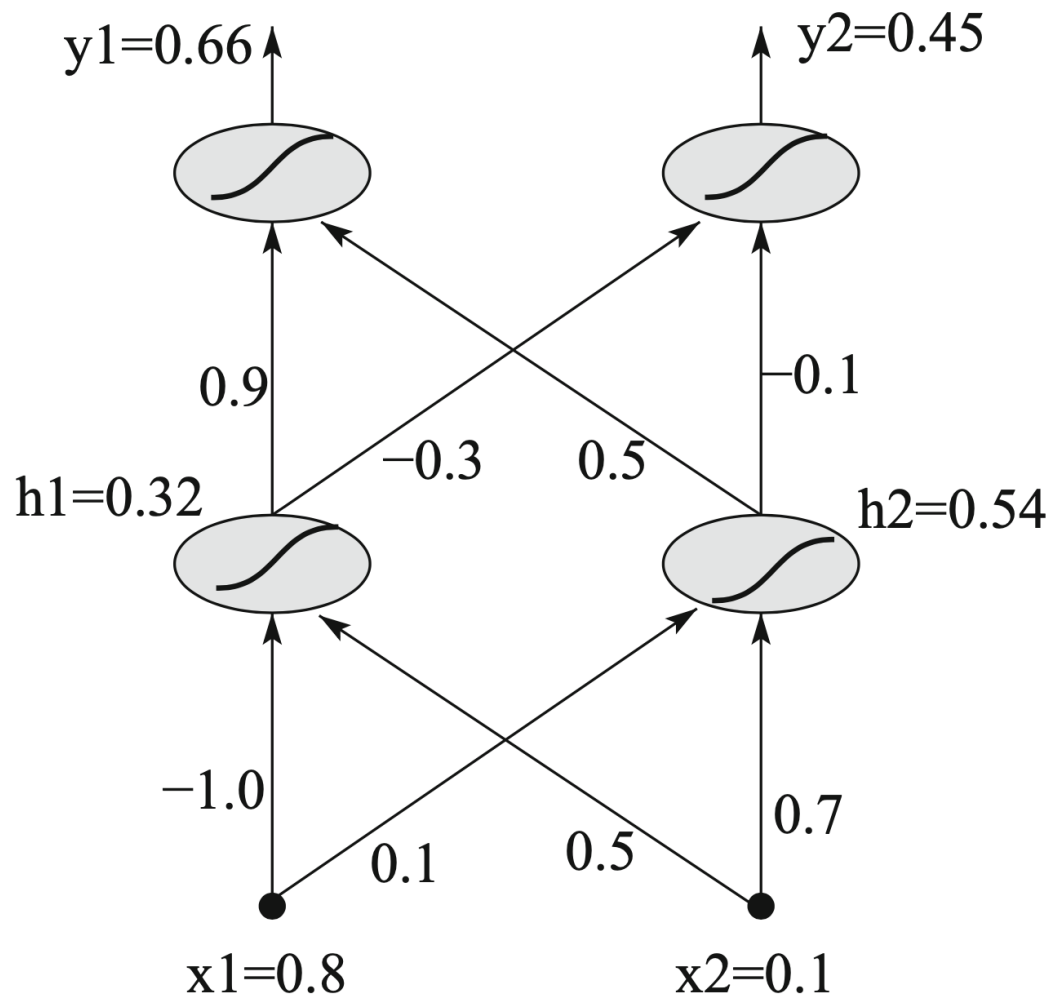
Multilayer Perceptron – Forward Propagation

- Input signal — a sample: $\mathbf{x} = \langle x_1, \dots, x_n \rangle$
 - ▢ delivered to the network inputs
- Processing by the hidden layer — for each j -th hidden neuron
$$f(\sum_k w_{kj} x_{kj} + b_j)$$
- Processing by the output layer — for each i -th neuron
$$y_i = f(\sum_j w_{ji} f(\sum_k w_{kj} x_{kj} + b_j) + b_i)$$
 - ▢ each output neuron receives outputs from all hidden neurone
- For $\mathbf{x}$ the network output $\mathbf{y}$ depends on
 - ▢ the network architecture
 - ▢ weights and biases
- Normalised output
 - ▢ softmax layer — converting raw output values into probabilities

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^d \exp(\mathbf{z}_j)} \quad 1 \leq i \leq d$$

(Jurafsky & Martin, pp. 141, draft Jan 2022)

MLP – Forward Propagation – Example



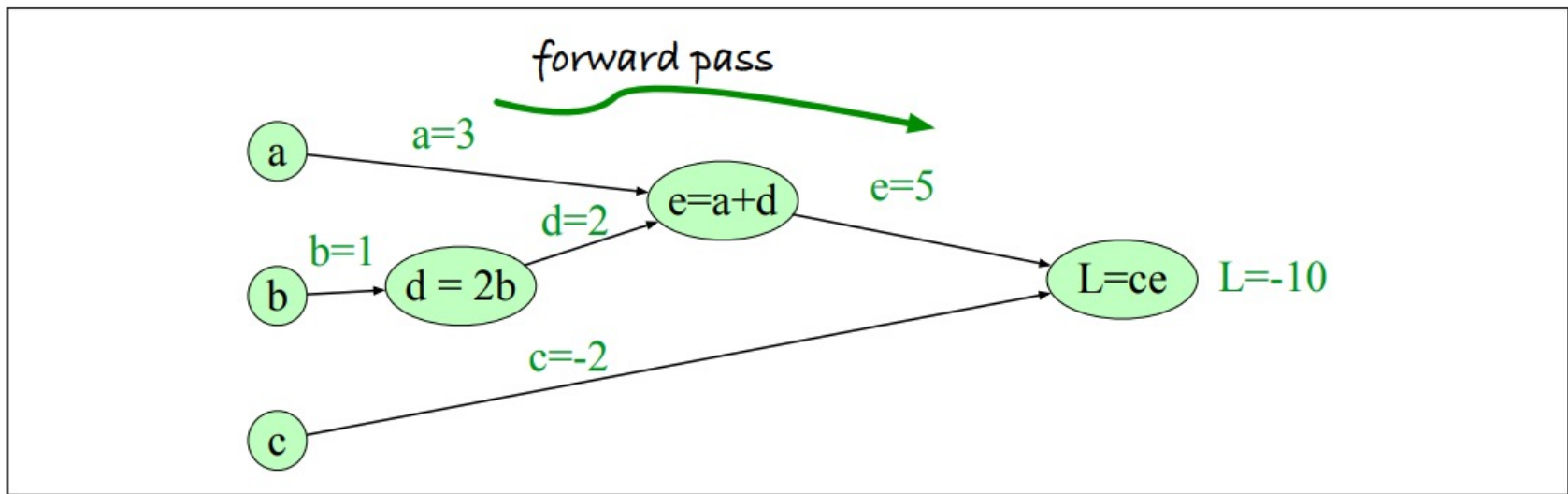
(Kubot, pp. 94, 2017)

Multilayer Perceptron – Training

- Input signal — a sample: $\mathbf{x} = \langle x_1, \dots, x_n \rangle$
 - delivered to the network inputs
 - Target vector $\mathbf{t}(\mathbf{x})$ — expected values of the output nodes
 - e.g. $t_i(\mathbf{x}) = 1$ and $t_j(\mathbf{x}) = 0$ for all $j \neq i$ — may encode i -th class in classification
 - Learning process
 - gradually adjusting the weights and biases
 - to make the outputs matching the training signals — the target vectors
 - * training samples: $\langle \mathbf{x}_1, \mathbf{t}(\mathbf{x}_1) \rangle, \langle \mathbf{x}_2, \mathbf{t}(\mathbf{x}_2) \rangle, \dots$
 - Network error (loss, loss function)
 - e.g. Mean Square Error
- $$MSE = \frac{1}{m} \sum_{i=1}^m (t_i - y_i)^2$$
- Main question: how to make this happen?
 - The Gradient Descent
 - changing weights towards steepest descent along the error function

Computational graph

- A computation graph is a graph representing the process of mathematical operations in neural network, where each operation is modeled as a node in a graph.



(Jurafsky & Martin, pp. 152, draft Jan 2023)

Chain rule

- The importance of computational graph comes from the **backward pass**, which is used to compute the derivatives needed for the network weights update.
- Let's assume we have $f(x) = u(v(x))$, when computing derivative we get:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

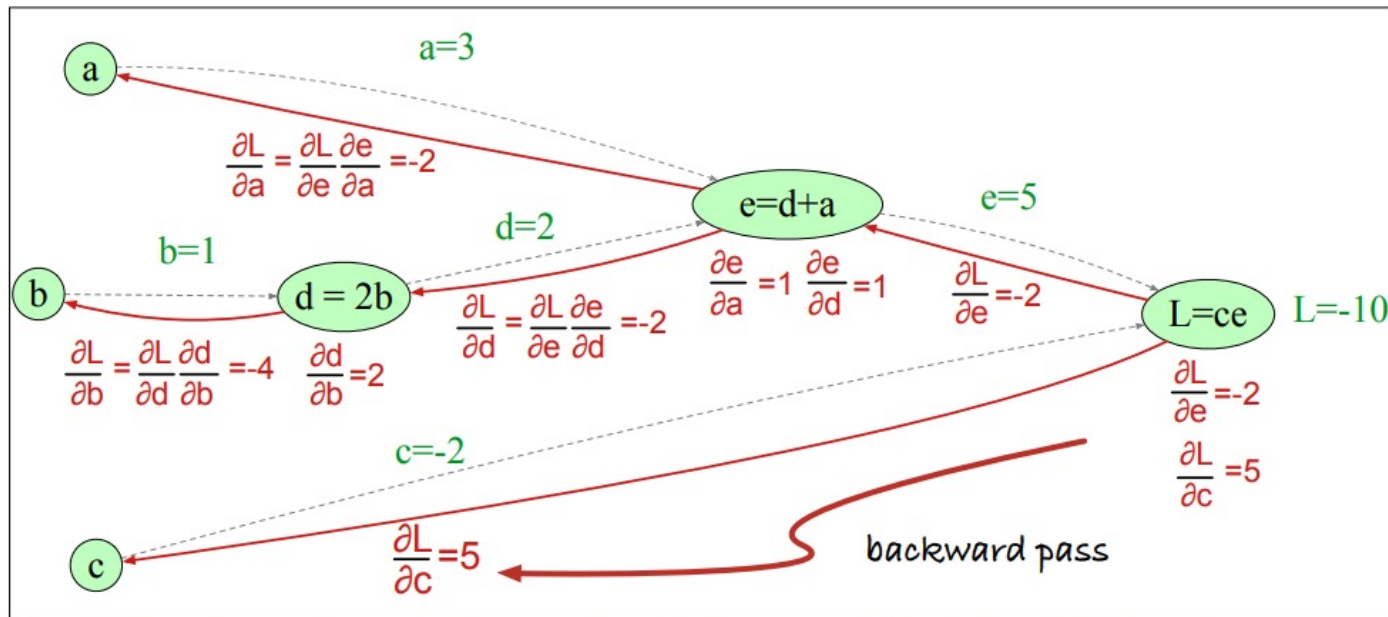
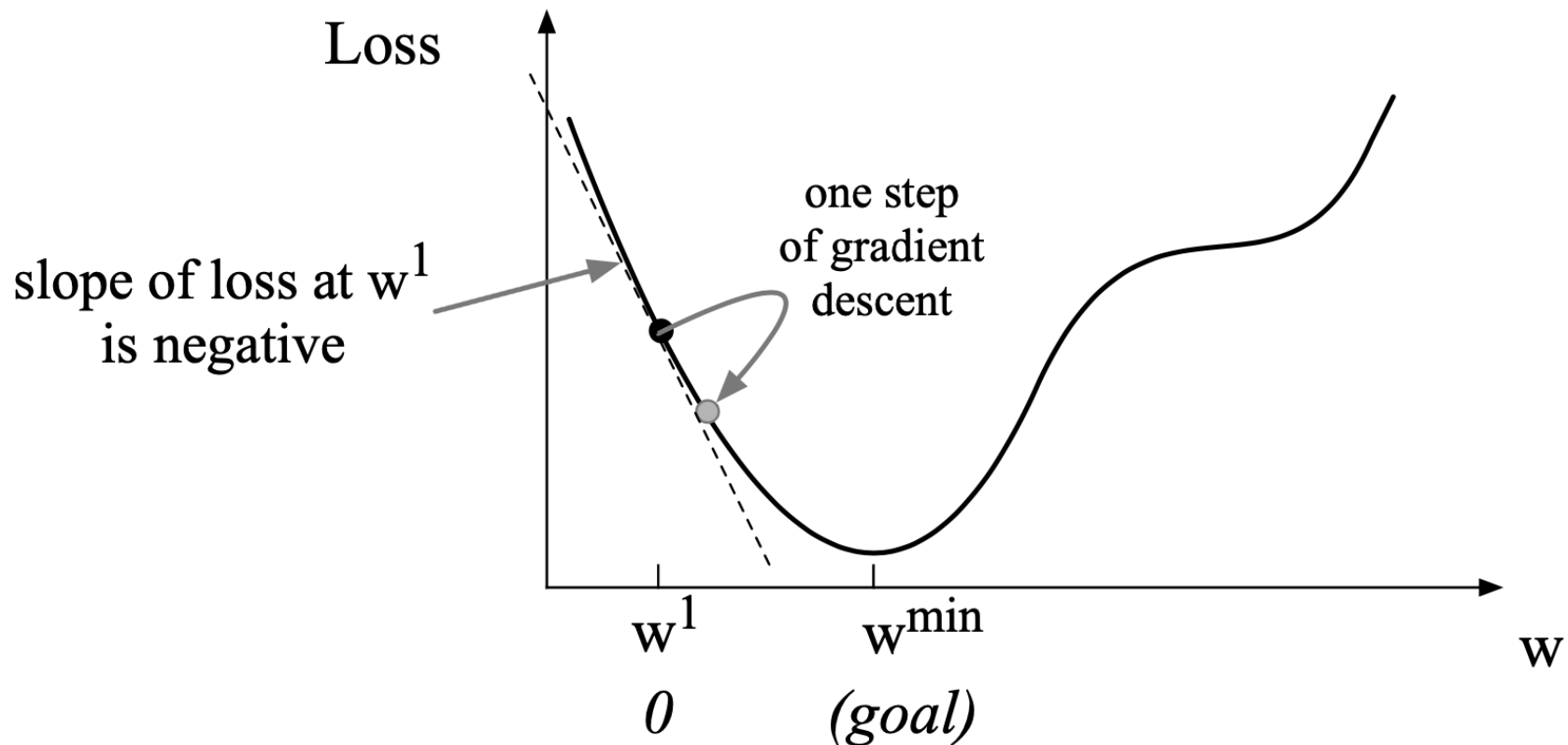


Figure 7.16 Computation graph for the function $L(a, b, c) = c(a + 2b)$, showing the backward pass computation of $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$.

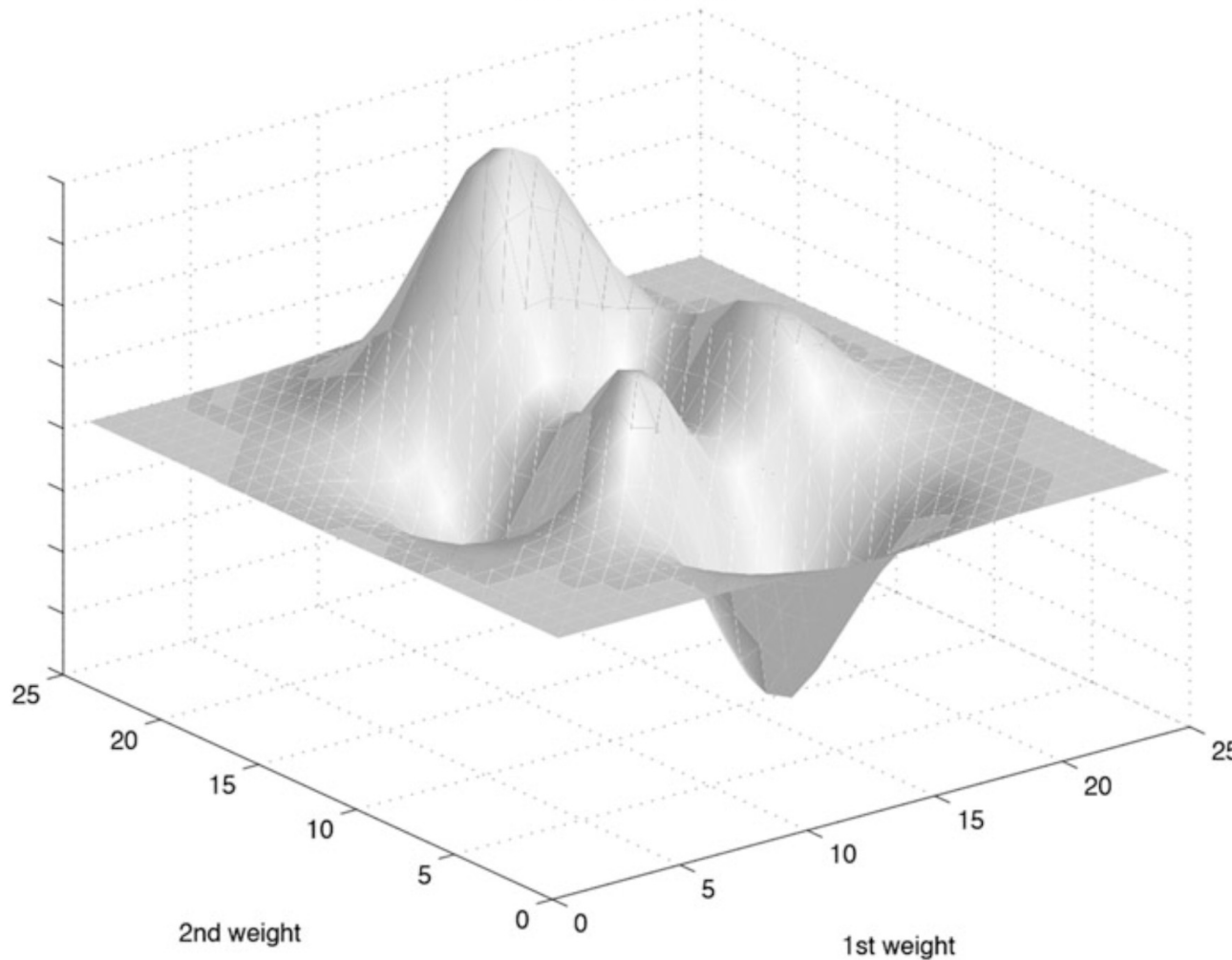
Backpropagation of Error – Gradient Descent

(Jurafsky & Martin, pp. 91, draft Jan 2023)



Backpropagation of Error – Gradient Descent

mean squared error



(Kubot, pp. 97, 2017)

- Towards lowest minimum
 - $$W_{\text{updated}} = W_{\text{old}} - \eta \nabla E$$
 - $$w \leftarrow w - \eta \nabla E$$
- (Skansi, pp. 94, 2019)

Backpropagation of Error

Table 5.2 *Backpropagation of error* in a neural network with one hidden layer

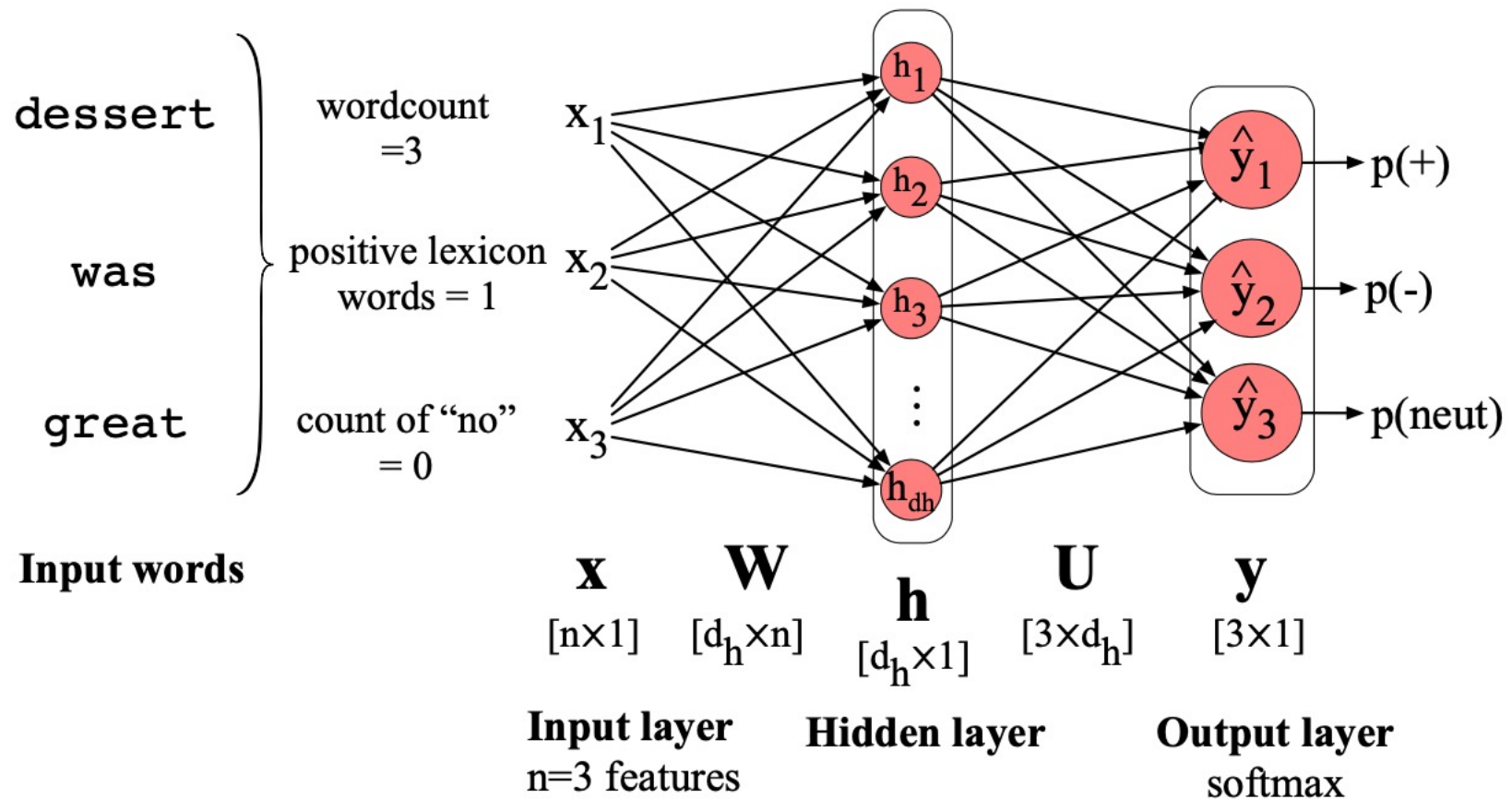
1. Present example $\mathbf{x}$ to the input layer and propagate it through the network.
2. Let $\mathbf{y} = (y_1, \dots, y_m)$ be the output vector, and let $\mathbf{t}(\mathbf{x}) = (t_1, \dots, t_m)$ be the target vector.
3. For each output neuron, calculate its responsibility, $\delta_i^{(1)}$, for the network's error:

$$\delta_i^{(1)} = y_i(1 - y_i)(t_i - y_i)$$
4. For each hidden neuron, calculate its responsibility, $\delta_j^{(2)}$, for the network's error. While doing so, use the responsibilities, $\delta_i^{(1)}$, of the output neurons as obtained in the previous step.

$$\delta_j^{(2)} = h_j(1 - h_j) \sum_i \delta_i^{(1)} w_{ji}$$
5. Update the weights using the following formulas, where η is the learning rate:
 output layer: $w_{ji}^{(1)} := w_{ji}^{(1)} + \eta \delta_i^{(1)} h_j$; h_j : the output of the j -th hidden neuron
 hidden layer: $w_{kj}^{(2)} := w_{kj}^{(2)} + \eta \delta_j^{(2)} x_k$; x_k : the value of the k -th attribute
6. Unless a termination criterion has been satisfied, return to step 1.

- (Kubot, pp. 100, 2017)
- See also: <https://medium.com/@serbanliviu/backpropagation-step-by-step-c8062593db27>

Example: MLP-based text classification



(Jurafsky & Martin, pp. 144, draft Jan 2022)

Overfitting

Neural networks due to large number of parameters are prone to overfitting, the model fits to the training data set but does not perform well on the test set.

The most obvious ways to deal with overfitting problem is to:

- **Increase training data:** Gathering more diverse and representative data can help the model learn better patterns and reduce overfitting. If it's feasible, collecting additional data can be an effective solution.
- **Reduce model complexity:** Overly complex models, such as those with a large number of parameters or deep neural networks, are more prone to overfitting.
- **Data augmentation:** Augmenting the training data by applying transformations, for example in case of image data: rotations, translations, or flips can help increase its diversity.
- **Noise** – random distortion added to the training data.

Regularization

- **Neural architecture and parameters** – build neural architecture for given problem and data domain. For example, use CNN for image classification. When training on new data
 - reuse already pretrained network,
 - and fine-tune only few last layers.
- **Early stopping** – monitoring the error on the Dev Set and stopping the training procedure when the error begins to rise.
- **Trade off between breadth and depth** – neural network with more layers tend to require far fewer neurons per layer because of the composition functions created by successive layers. This makes possible for network to have less parameters overall
 - at least two layers for modelling functions of any shape
- **Dropout/Dropconnect** – randomly selected units/connections are not used in the training iteration and excluded from error propagation. This leads to more expressive and distinct features

Controlling training (optimisation)

- Learning factor
- Momentum
 - $w_{t+1} = w_t - \alpha m_t$, where m_t is moving average of the gradients

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

- RMSprop (Root Mean Square Propagation)

$$w_{t+1} = w_t - \frac{\alpha_t}{\sqrt{v_t + \epsilon}} \frac{\partial L}{\partial w_t}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

<https://www.geeksforgeeks.org/adam-optimizer/>

Controlling training

Adam Optimiser

- First moment (mean) estimate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

- Second moment (variance) estimate:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial w_t} \right)^2$$

- Bias correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

- Final weight update

$$w_{t+1} = w_t - \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \alpha$$

- Parameters (typically): $\alpha=0.001$, $\beta_1=0.9$ and $\beta_2=0.999$, $\epsilon=10^{-8}$

<https://www.geeksforgeeks.org/adam-optimizer/>

Vanishing and exploding gradient

- **Exploding Gradient** - when the gradients during backpropagation become extremely large, it is known as the exploding gradient problem. As the gradients are backpropagated through the network, they can multiply together, resulting in exponentially increasing values. This can cause the weights to update dramatically, leading to unstable training and making it difficult for the model to converge. In extreme cases, the weights may become NaN (not a number) or infinity.
- **Vanishing Gradient** – on the other hand, the vanishing gradient problem occurs when the gradients during backpropagation become extremely small. As the gradients are backpropagated through the network, they can diminish exponentially, particularly in deep networks with many layers. When the gradients become too small, the weight updates are minimal, and the model may fail to learn complex patterns effectively. The early layers of the network are most affected by the vanishing gradient problem.

Vanishing and exploding gradient – solutions (1)

- **Gradient clipping:** This technique involves scaling down the gradients if their norm exceeds a certain threshold. It helps to prevent the exploding gradient problem by limiting the magnitude of the gradients.
- **Batch normalization:** normalizes the activations within each mini-batch during training. It helps to mitigate the exploding and vanishing gradient problems by maintaining a more stable distribution of activations and gradients.
- **Initialization strategies:** Proper initialization of the weights can alleviate the vanishing gradient problem. Techniques like Xavier or Kaiming He initialization set the initial weights in a way that balances the variance of activations and gradients throughout the network.

Vanishing and exploding gradient – solutions (2)

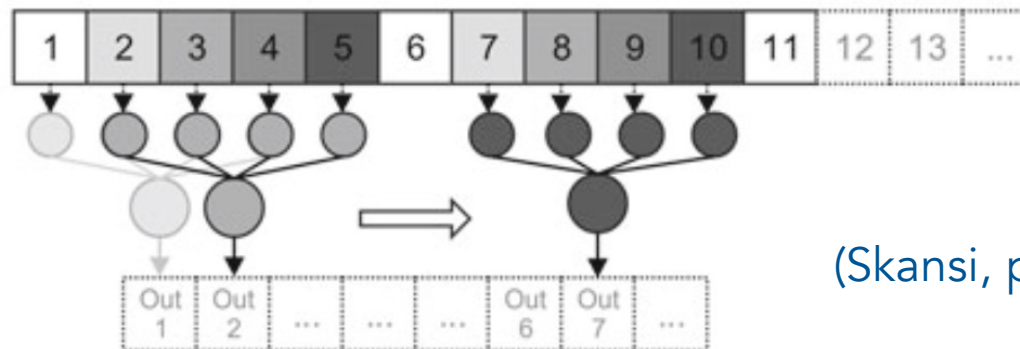
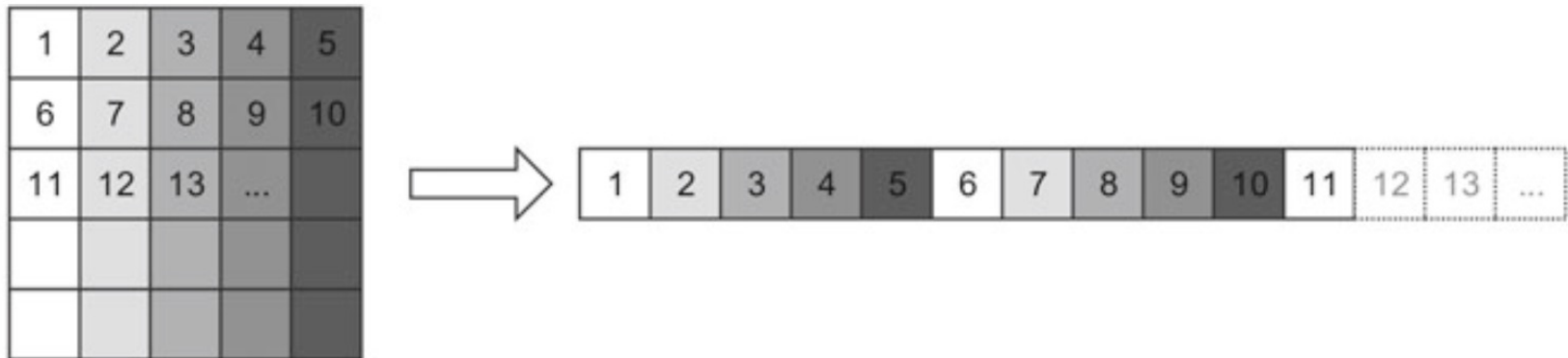
- **Non-saturating activation functions:** ReLU (Rectified Linear Unit) and its variants are less prone to the vanishing gradient problem compared to sigmoid or tanh functions. They allow gradients to flow more easily through the network.
- **Neural network modifications:** using residual connections, which allow gradients to propagate more easily or special types of recurrent networks: LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit).

Epoch, batch and minibatch

- Epoch is a complete pass through the entire training dataset.
- Batch gradient descent is an optimizing step over all training instances. This leads to global optimum, but it is not used in practice as it is very slow to compute. That is why in practice **batch size** is a size of a **minibatch**.
 - gradients = average gradients across the batch samples
 - batch size: $s=1$ (gradient descent), $s=m$ (mini-batch), $s=n$ (batch, full batch)
- Minibatch is related to **stochastic gradient descent (SGD)**, where a small number of training examples is taken at each step to perform a network update. This leads to more frequent updates, but it can't be guaranteed they lead towards global optimum.

Convolutional Neural Networks

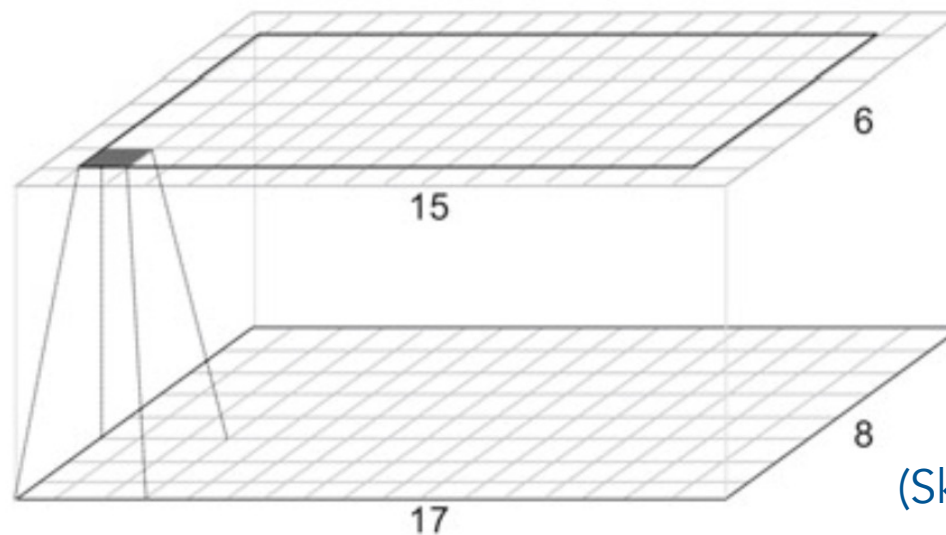
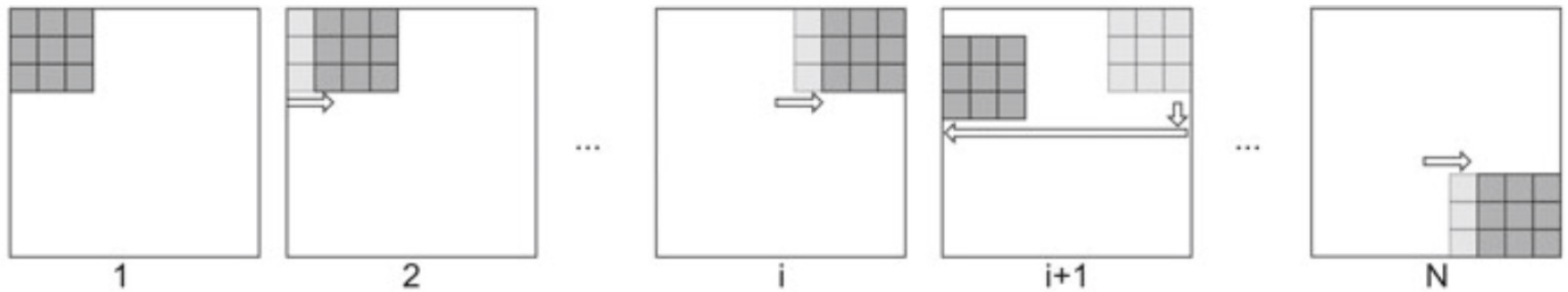
- The idea of convolution
 - ▢ receptive fields in biology
 - ▢ dimensionality reduction by focusing on small sub-areas
- 1D convolution



(Skansi, pp. 122, 2017)

Convolutional Neural Networks

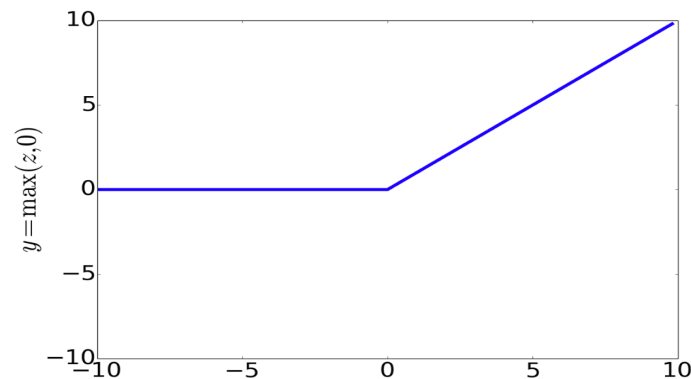
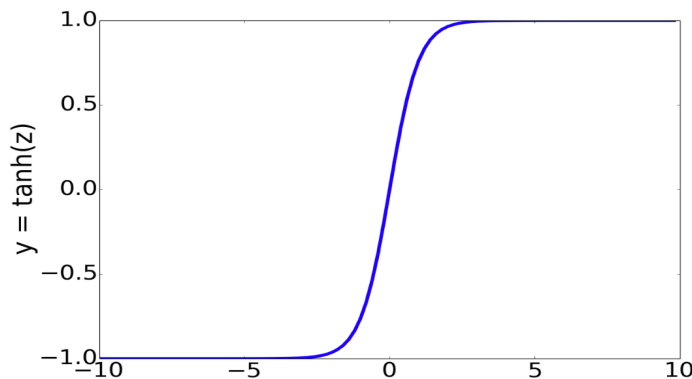
- 2D convolution — often used in image processing



(Skansi, pp. 123, 2017)

Convolutional Neural Networks

- CNN — typically Deep Neural Networks
 - with multiple hidden layers
- ReLU — rectified linear unit — cut-off linear transfer function
 - $\text{ReLU}(x) = \max(f(\sum_k w_{kj} x_{kj} + b_j), 0)$



- Feature Maps and Pooling
 - feature maps — define shape of areas from which signal is collected
 - * e.g. 2 x 2 (2D), 4 consecutive (1D), 4 sequence of every second (1D), etc.
 - pooling — methods for collecting signals into the output
 - * max pooling — maximum over an area (pick-one-out-of-all in the area)
 - * average pooling — average over the values in an area

(Jurafsky & Martin, pp. 136, draft Jan 2022)

ImageNet

- ImageNet is an image dataset organized according to the WordNet hierarchy. Each meaningful concept in WordNet, possibly described by multiple words or word phrases, is called a "synonym set" or "synset". There are more than 100,000 synsets in WordNet; the majority of them are nouns (80,000+). In ImageNet, we aim to provide on average 1000 images to illustrate each synset. Images of each concept are quality-controlled and human-annotated. In its completion, we hope ImageNet will offer tens of millions of cleanly labeled and sorted images for most of the concepts in the WordNet hierarchy.

Image classification

Easiest classes

red fox (100) hen-of-the-woods (100) ibex (100) goldfinch (100) flat-coated retriever (100)



tiger (100)

hamster (100)

porcupine (100)

stingray (100)

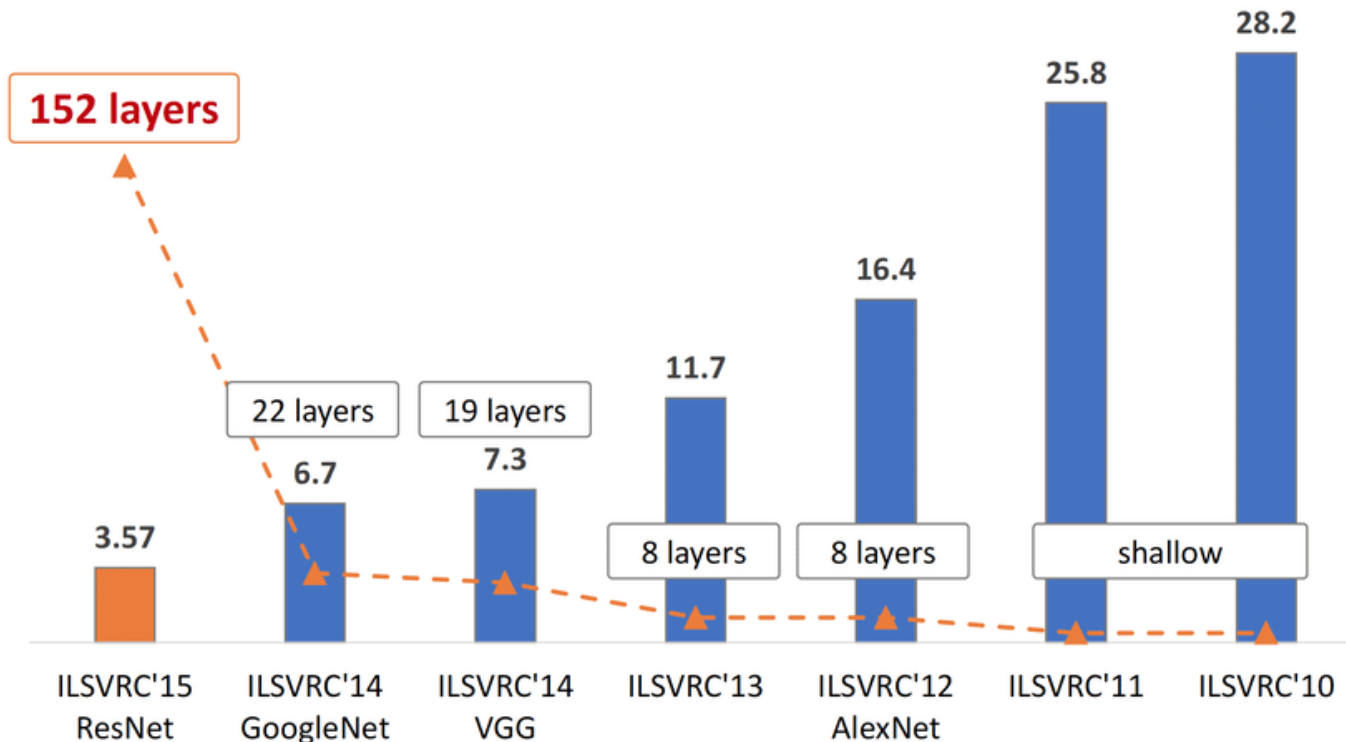
Blenheim spaniel (100)



ImageNet Large Scale Visual Recognition Challenge

ImageNet

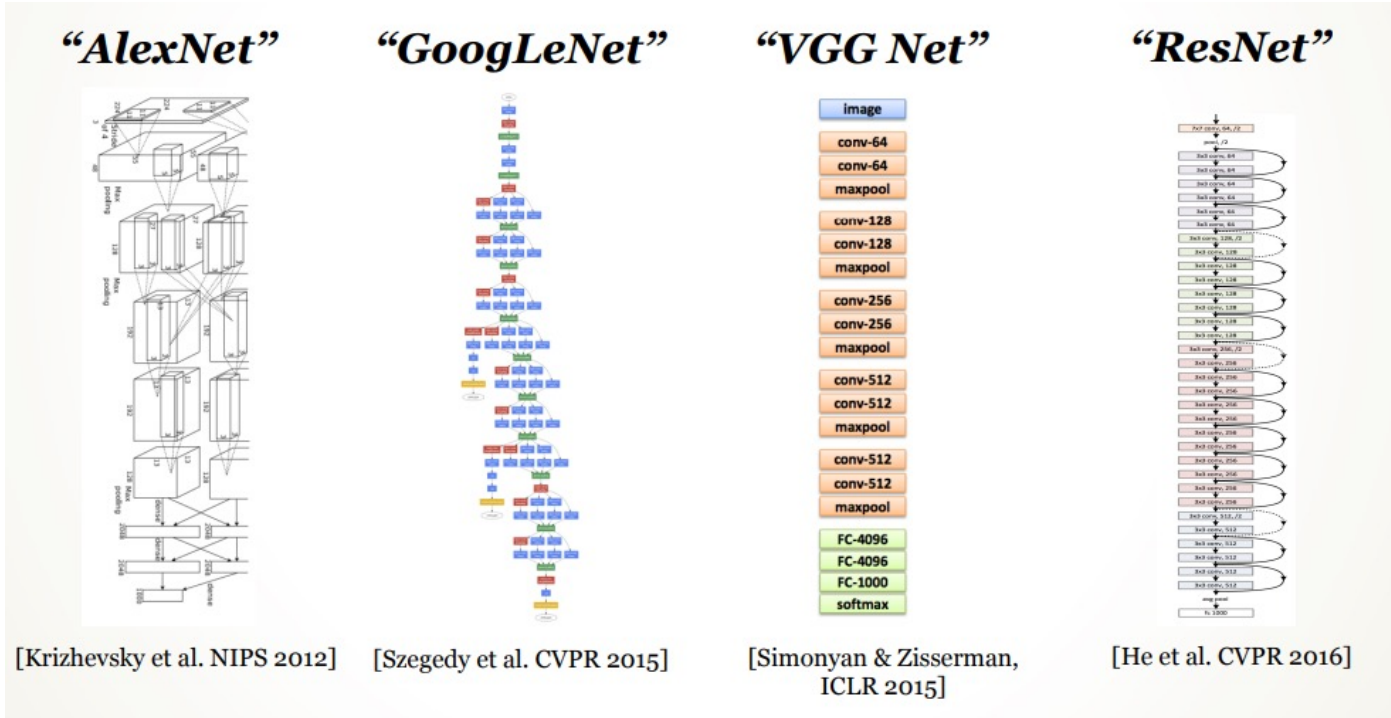
- Performance on ImageNet over the years, deep neural networks emerged and started to take over the computer vision



K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition, 2016"

ImageNet

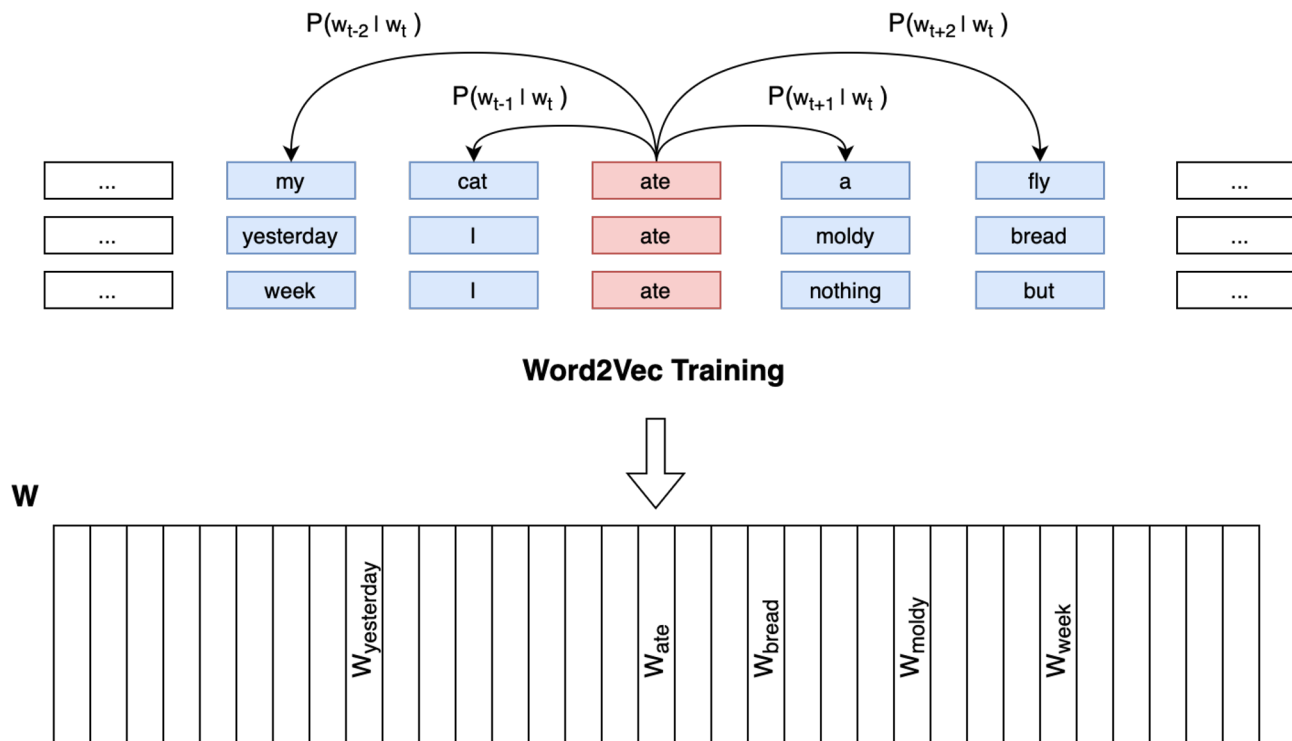
- Most notable architectures:



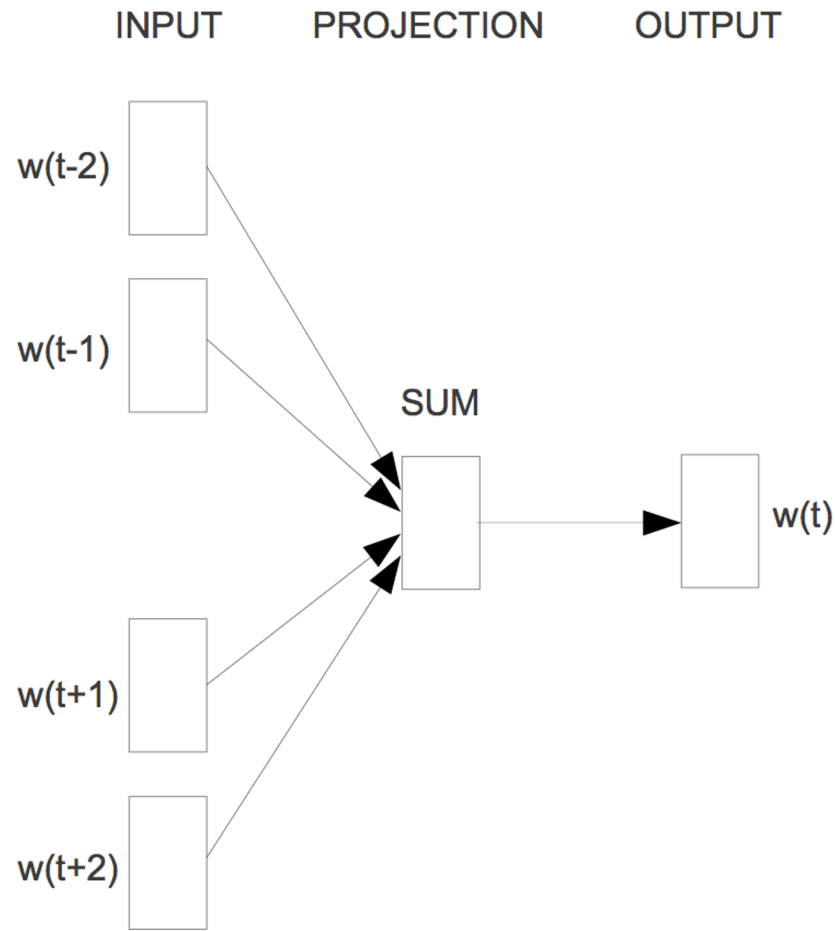
https://image-net.org/static_files/files/imagenet_ilsrvr2017_v1.0.pdf

Word Embeddings - Word Vector Models

- **LSA, Word2Vec, Fasttext** are algorithms for learning word representations
- The models resulting from these algorithms are not language models
- **Goal: to capture the semantic relationships between words**
 - reflecting semantic relations in algebraic terms

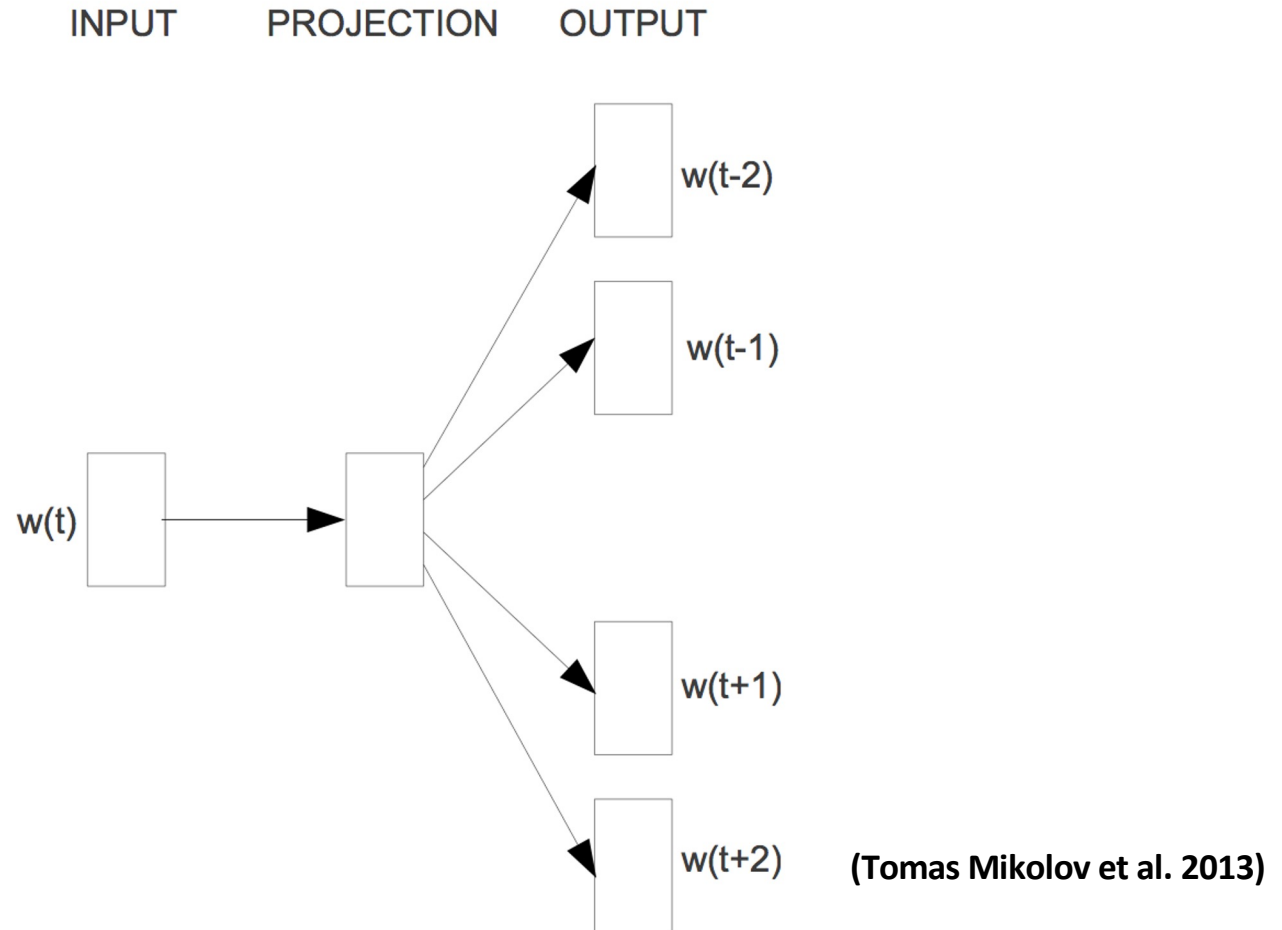


Word2vec – CBOW schema

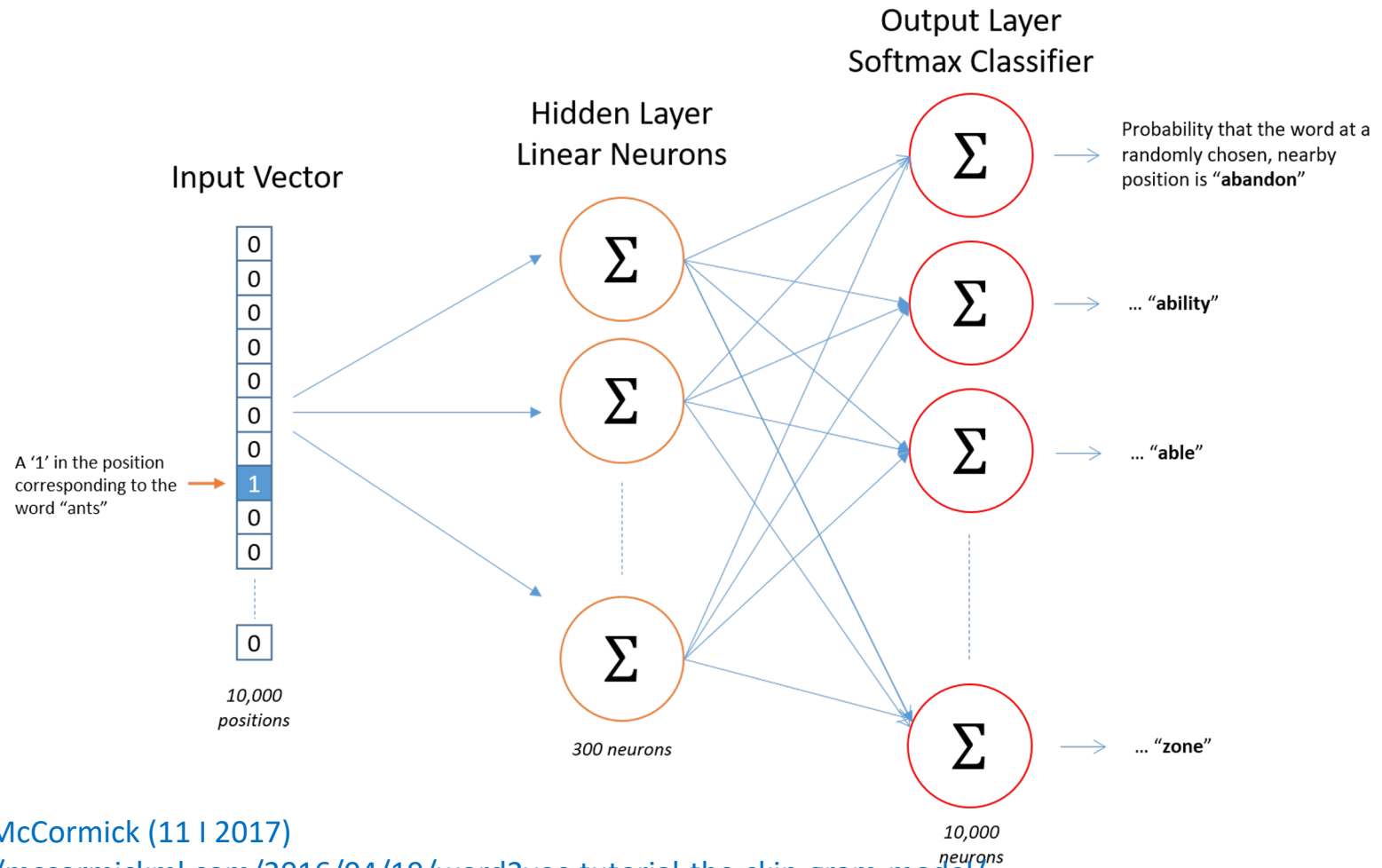


(Tomas Mikolov et al. 2013)

Word2vec – SkipGram schema



Word2vec – SkipGram schema



Chris McCormick (11 | 2017)

<http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/>

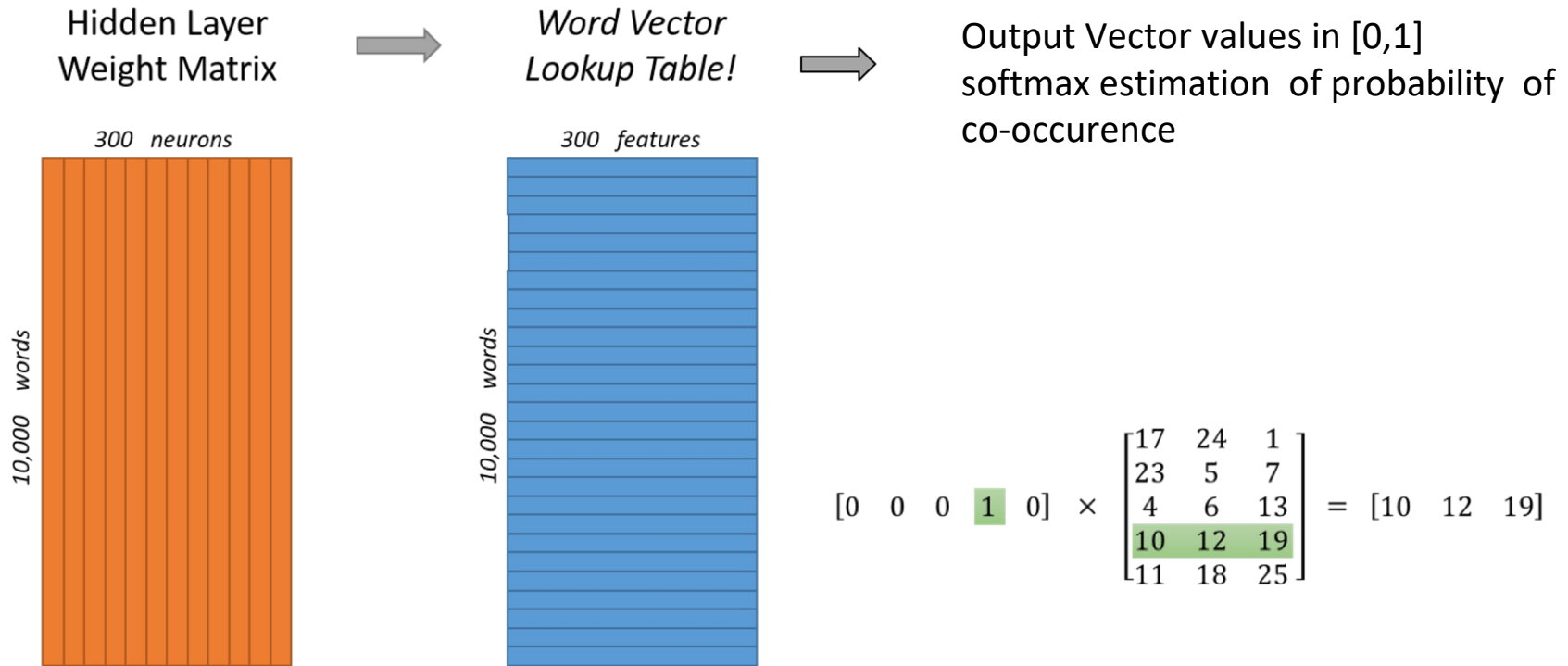
Word2vec – Training SkipGram

Source Text	Training Samples
The quick brown fox jumps over the lazy dog. →	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog. →	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog. →	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog. →	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

Chris McCormick (11 | 2017)

<http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/>

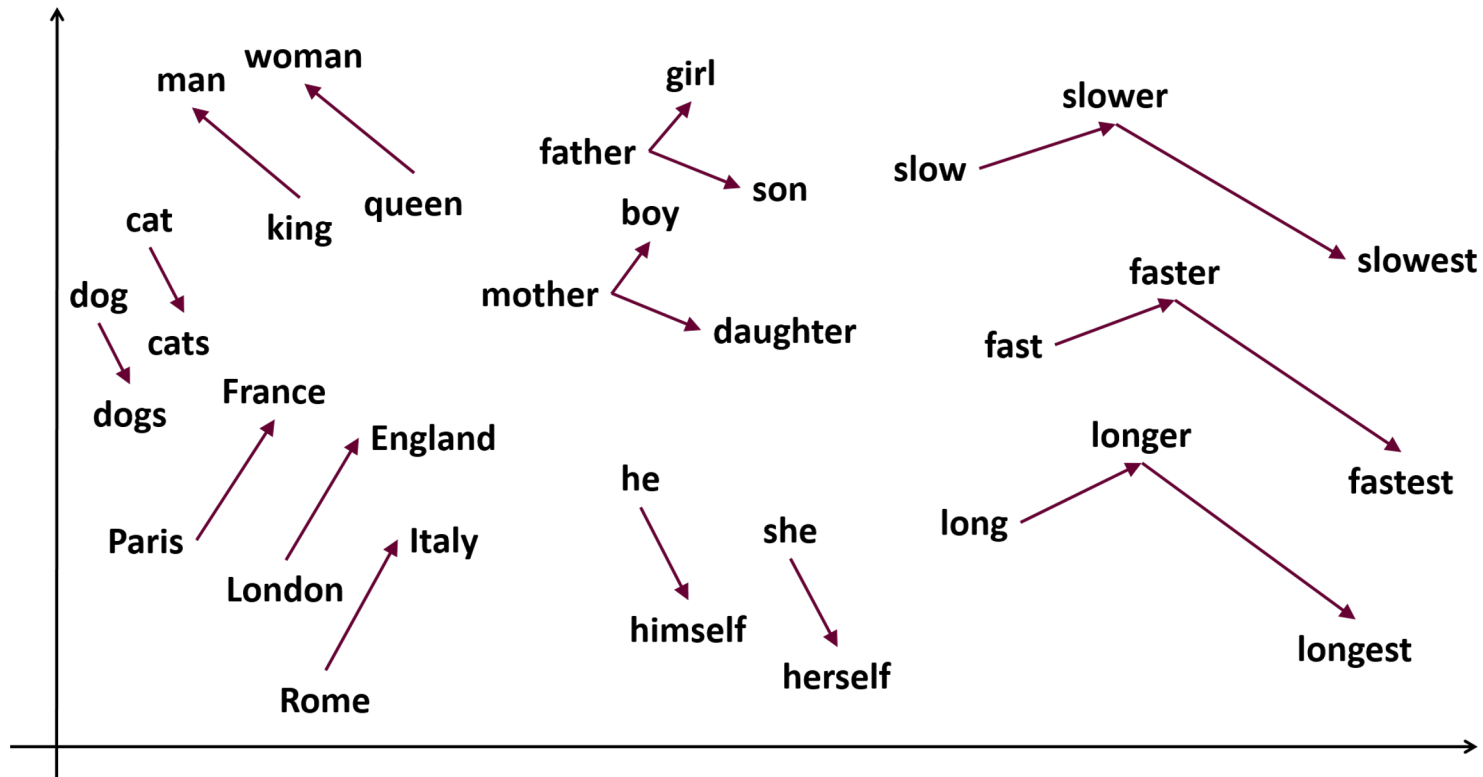
Word2vec – SkipGram at work



Chris McCormick (11 | 2017)

<http://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/>

Word Embeddings – Example of word similarities

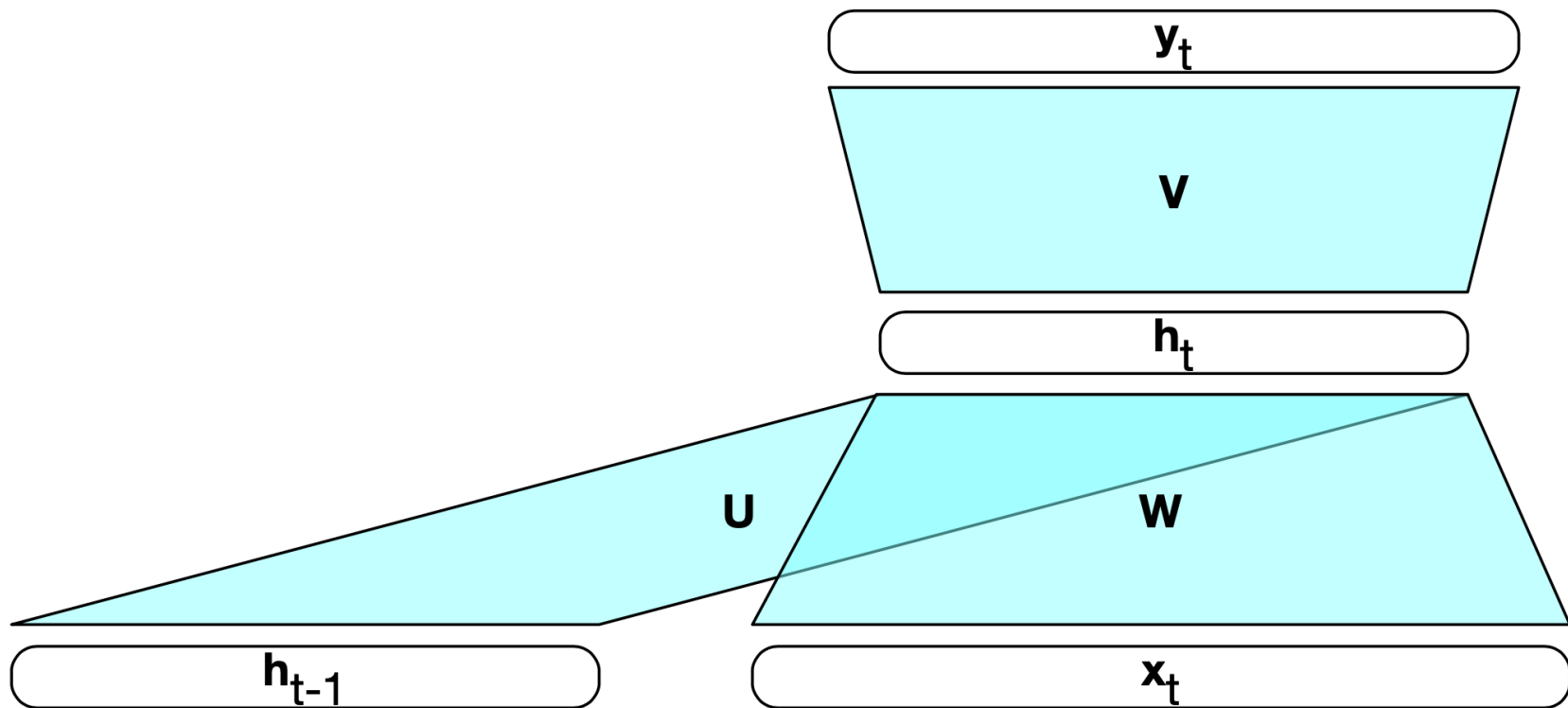


Source:

<https://www.samyza.com/ML/nlp/nlp.html>

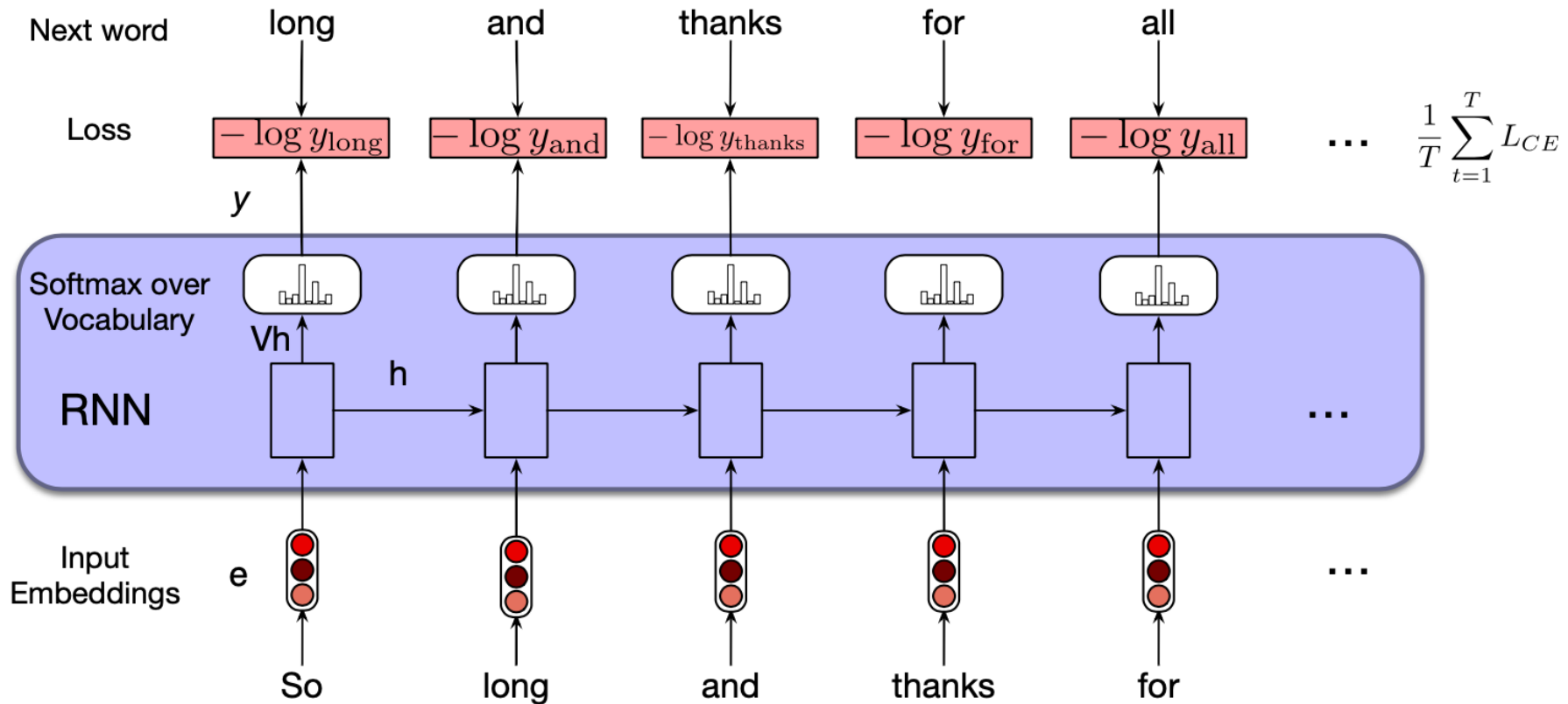
Recurrent Neural Networks

- Data larger than the neural network input (e.g. text stream)
- Simple Recurrent Neural Network
 - past hidden states as an additional input



(Jurafsky & Martin, pp. 187, draft Jan 2022)

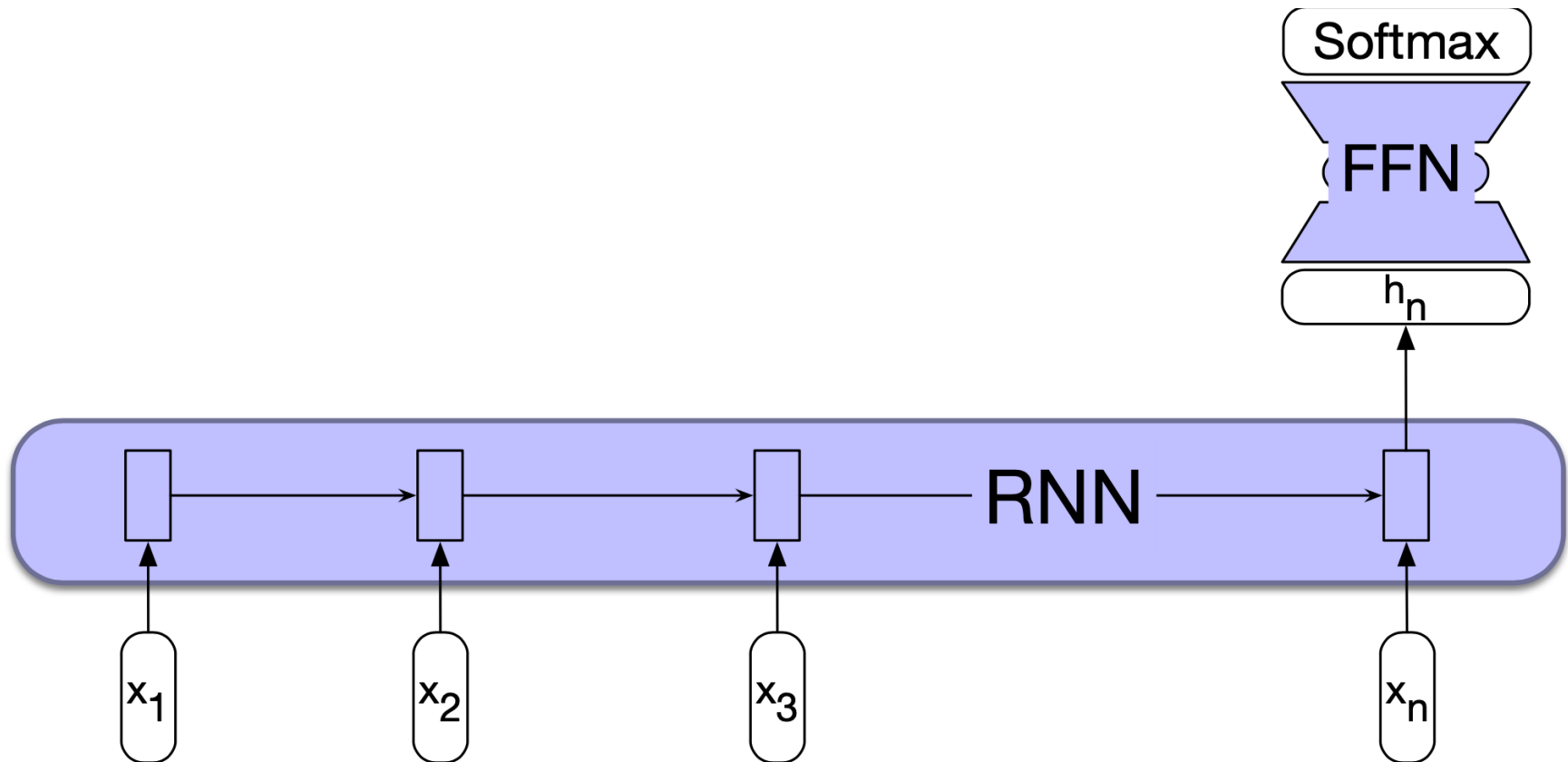
Example: RNN as a language model



(Jurafsky & Martin, pp. 191, draft Jan 2022)

- Loss (function) — error function to be minimised
- Embedding — vector representation of a word
 - one-hot encoding — one bit per word representation

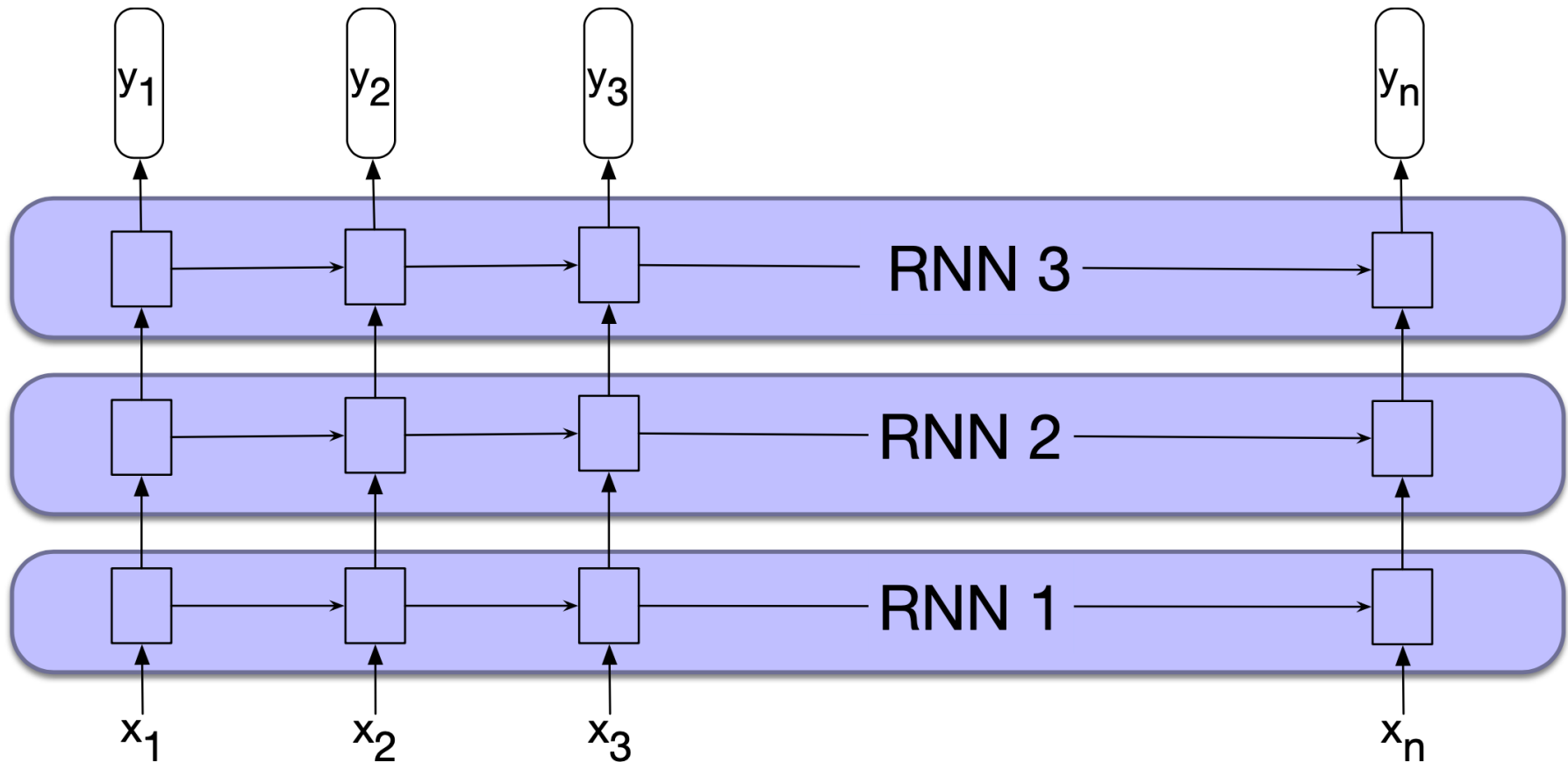
Example: RNN for sequence classification



(Jurafsky & Martin, pp. 193, draft Jan 2022)

- FFN — Feed Forward Network — MLP
- Last hidden layer of RNN as an signal representation (encoding)

Deep RNN

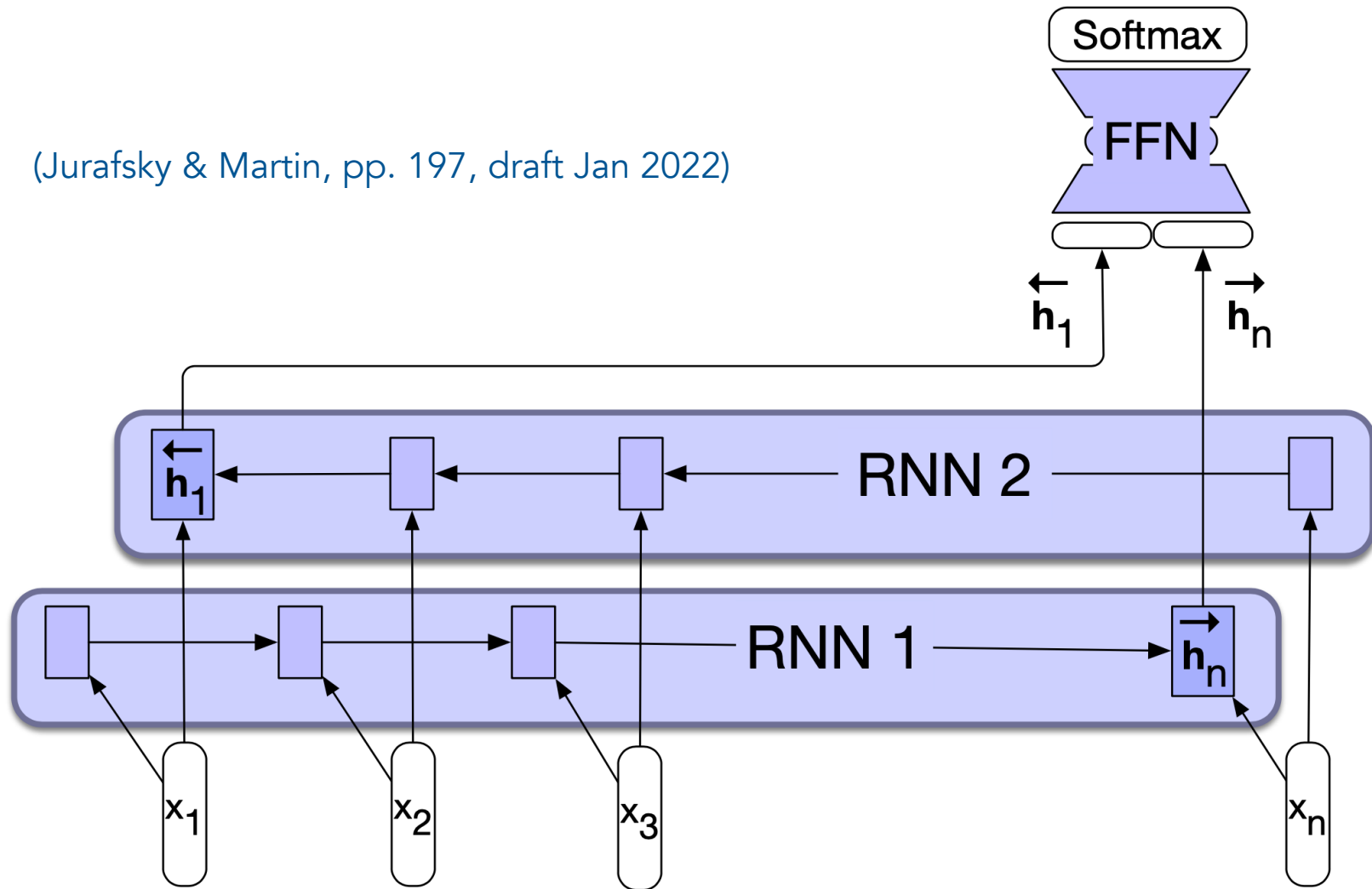


(Jurafsky & Martin, pp. 195, draft Jan 2022)

- More layers — larger capacity
 - danger of overfitting

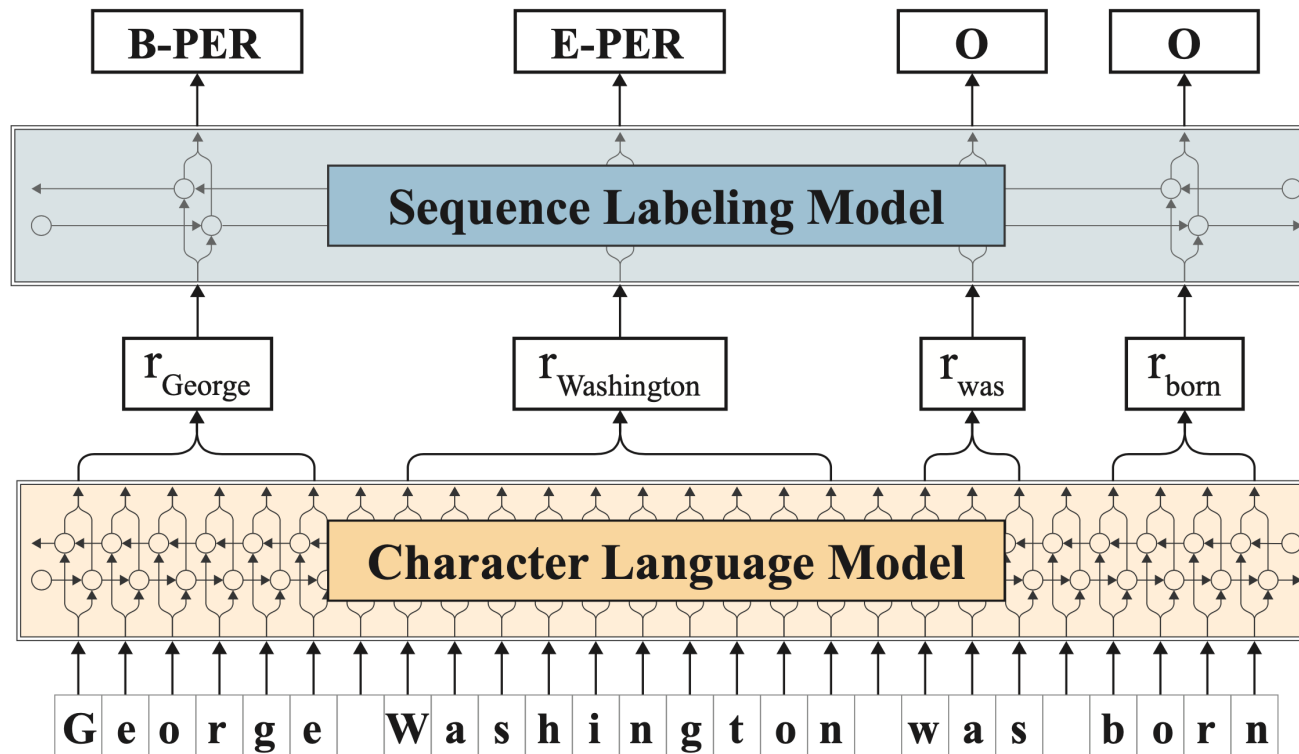
Bidirectional RNN for sequence classification

(Jurafsky & Martin, pp. 197, draft Jan 2022)



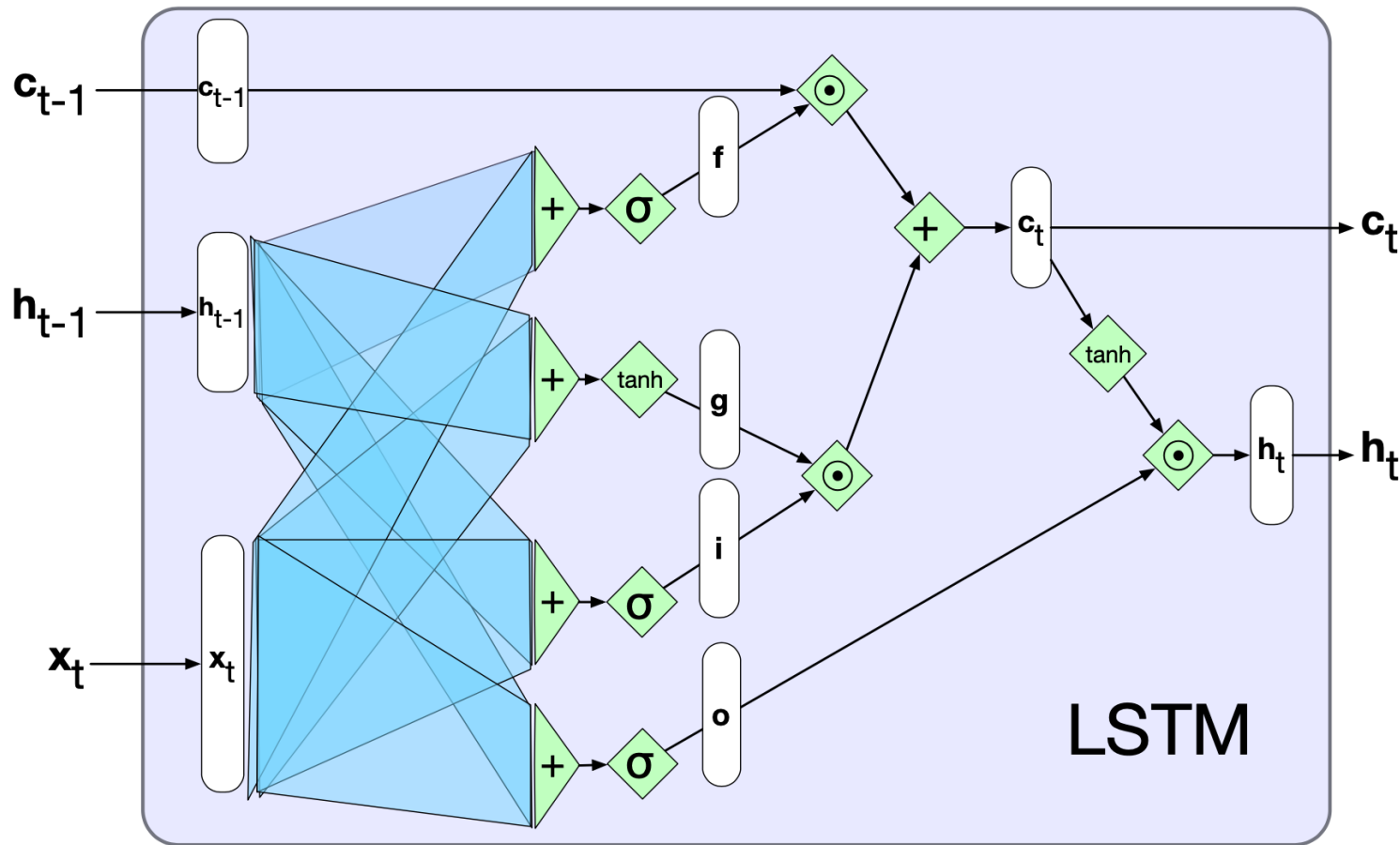
- Two directions — better picture of dependencies

Flair: example application



(Akbik et al., 2018), licencja CC

LSTM Networks – single unit

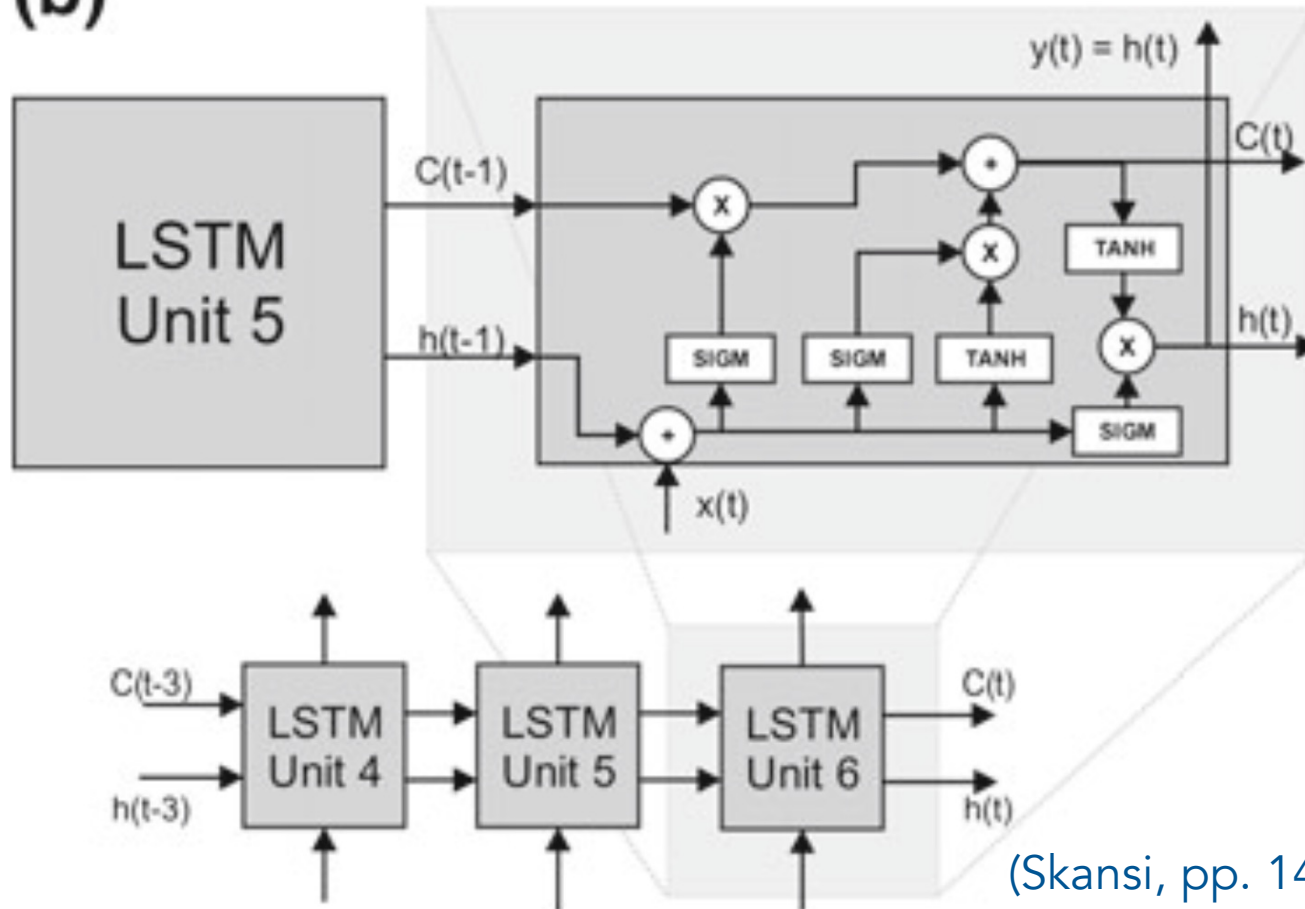


- LSTM — Long-Short Term Memory
- c_{t-1} — the previous context (cell state)
- c_t — the next context
- f - forget gate, g - add gate, o - output gate

(Jurafsky & Martin, pp. 199, draft Jan 2022)

LSTM Networks

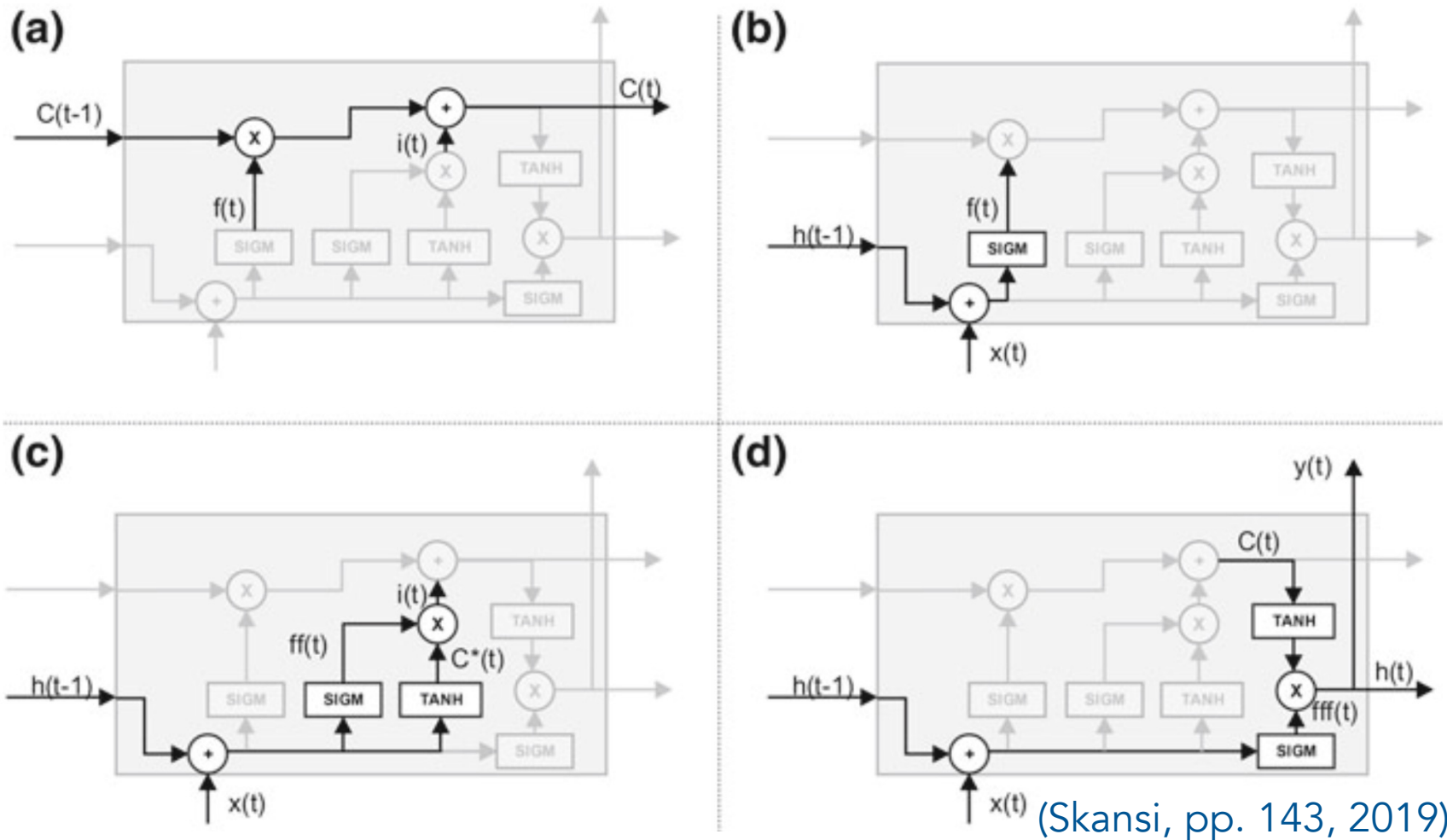
(b)



(Skansi, pp. 142, 2019)

- The long-short term dependencies are shaped inside cells

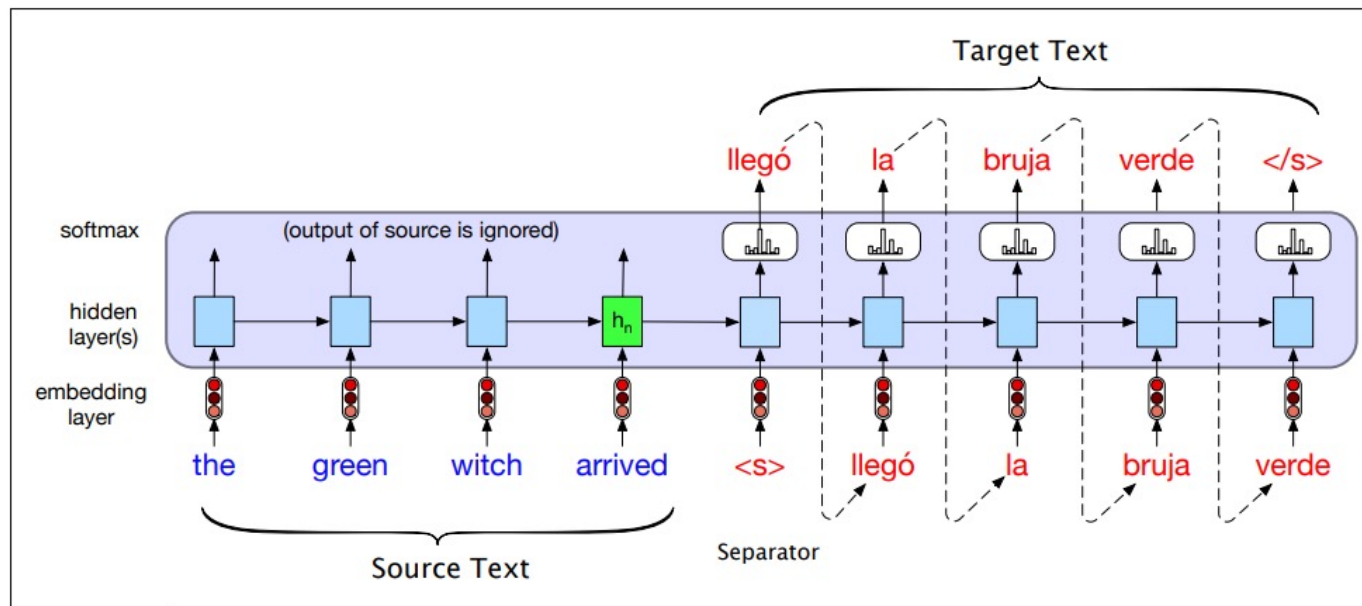
LSTM Networks



- a) cell state, b) forget gate, c) input gate, d) output gate

Encoder-decoder network

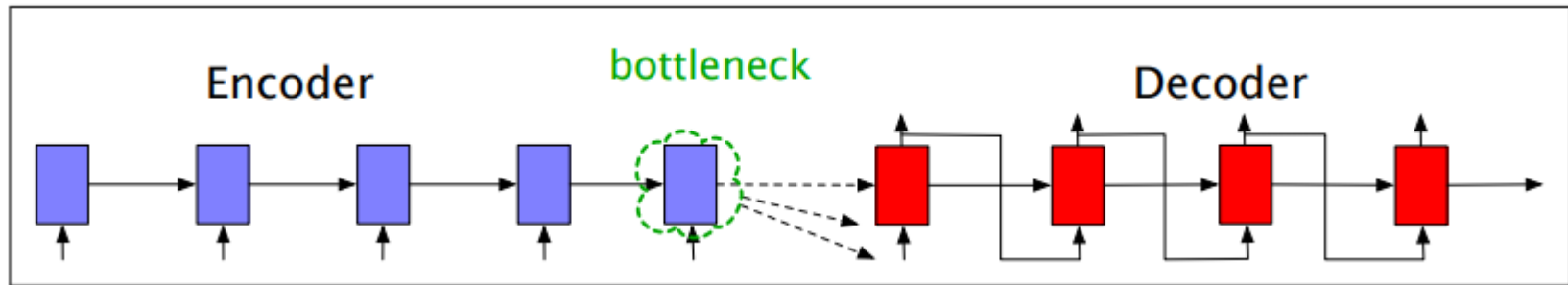
- Example of encoder-decoder network for machine translation.
- Encoder – accepts input sequence and generates contextualized representations
- Decoder – accepts context vector from encoder and generates an arbitrary length sequence



(Jurafsky & Martin, pp. 203, draft Jan 2023)

Attention mechanism

- The final hidden state of encoder must contain all the information about the input sentence, it must express all the meaning for the further decoding. It becomes a bottleneck.

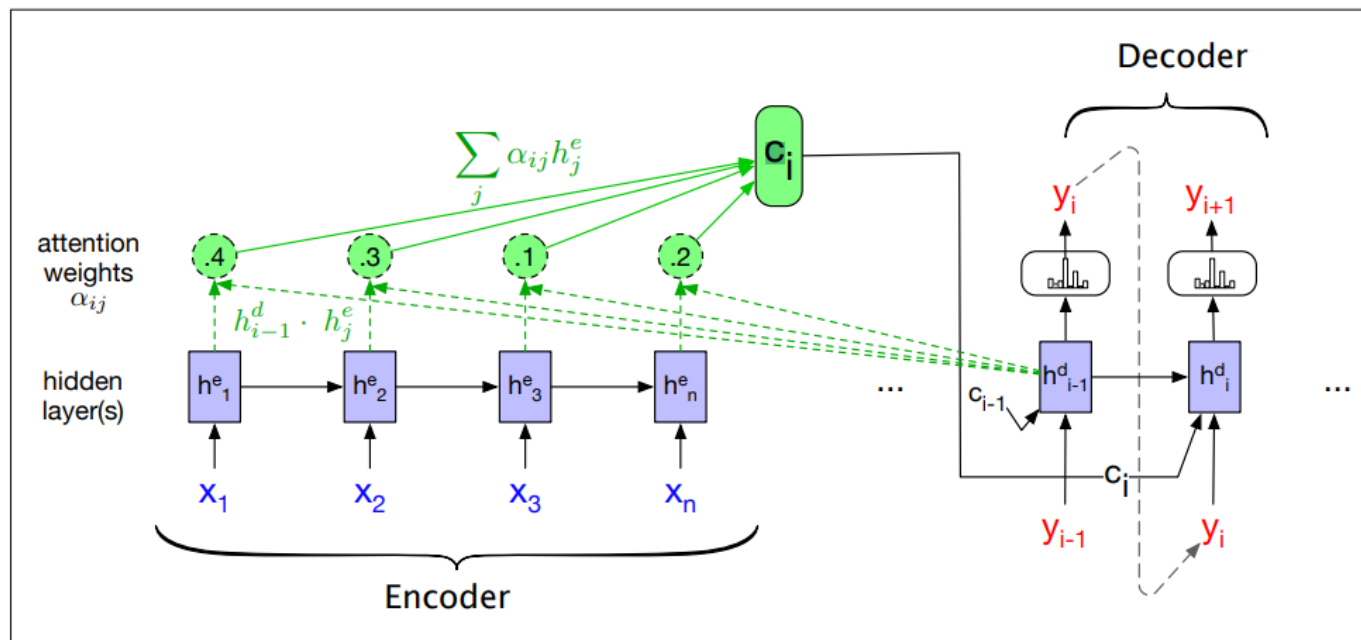


(Jurafsky & Martin, pp. 206, draft Jan 2023)

Attention mechanism

- The idea behind attention is to check how relevant each encoder state is to the decoder state.
- To score the relevance, dot-product can be applied. It measures the similarity between two hidden states and is called a **dot-product attention**

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$



(Jurafsky & Martin, pp. 208, draft Jan 2023)

Attention mechanism

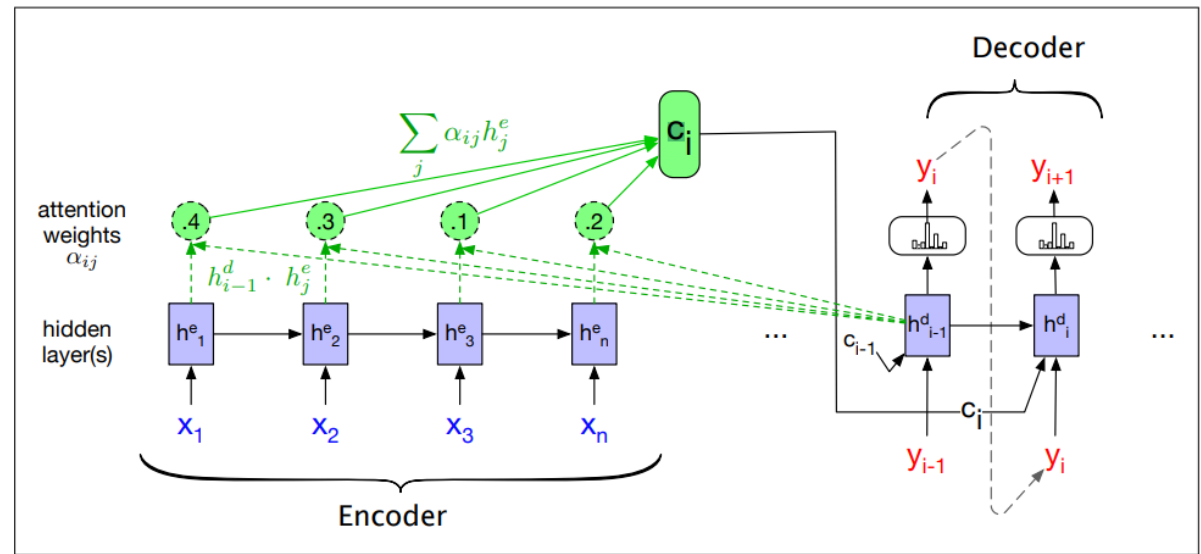
- Now, we have to compute scores over all encoder's hidden states and normalize with softmax function

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) \quad \forall j \in e)$$

$$= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}$$

- Afterwards context vector can be computed as weighted average over all encoder hidden states

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$



(Jurafsky & Martin, pp. 208, draft Jan 2023)

Transformers

- Models introduced in paper "Attention is all you need", where multi-head attention was introduced. It was a breakthrough in Natural Language Processing.
- Now, there are many different transformer architectures with different attentions and network structures. Worth mentions are: BERT, T5, BART, GPT 2, GPT3...

Attention Is All You Need, A. Vaswani, 2017

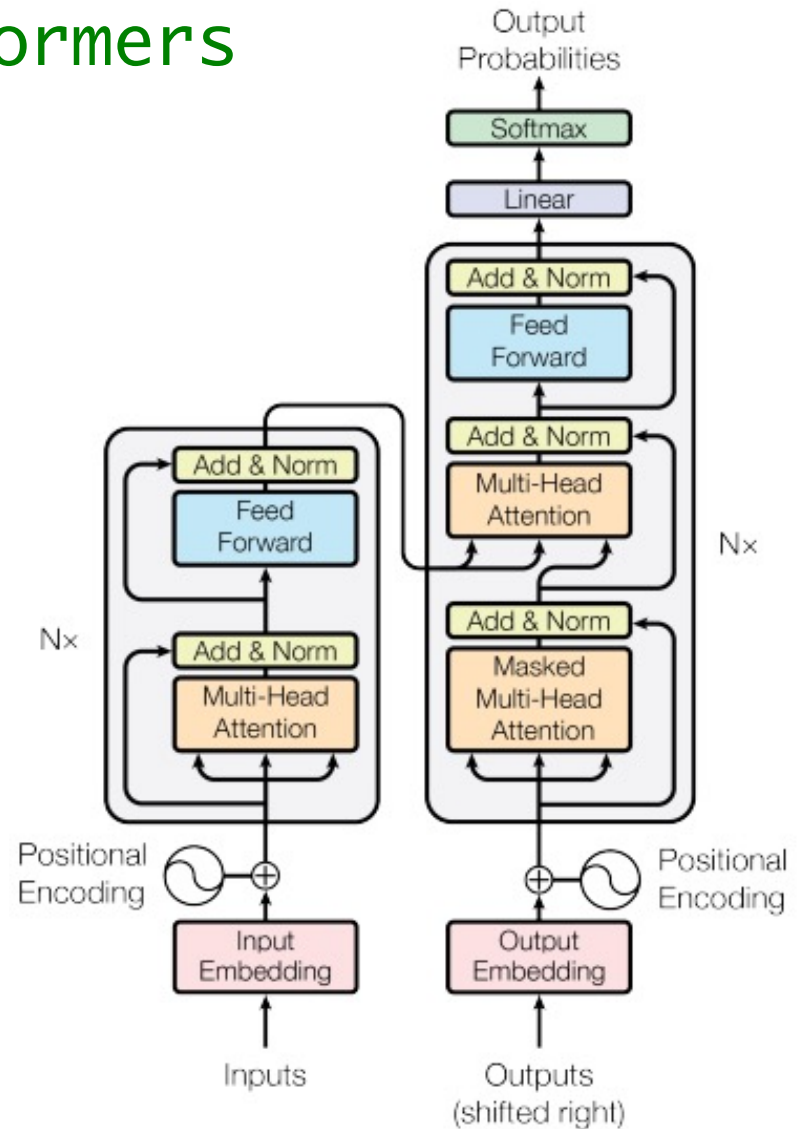
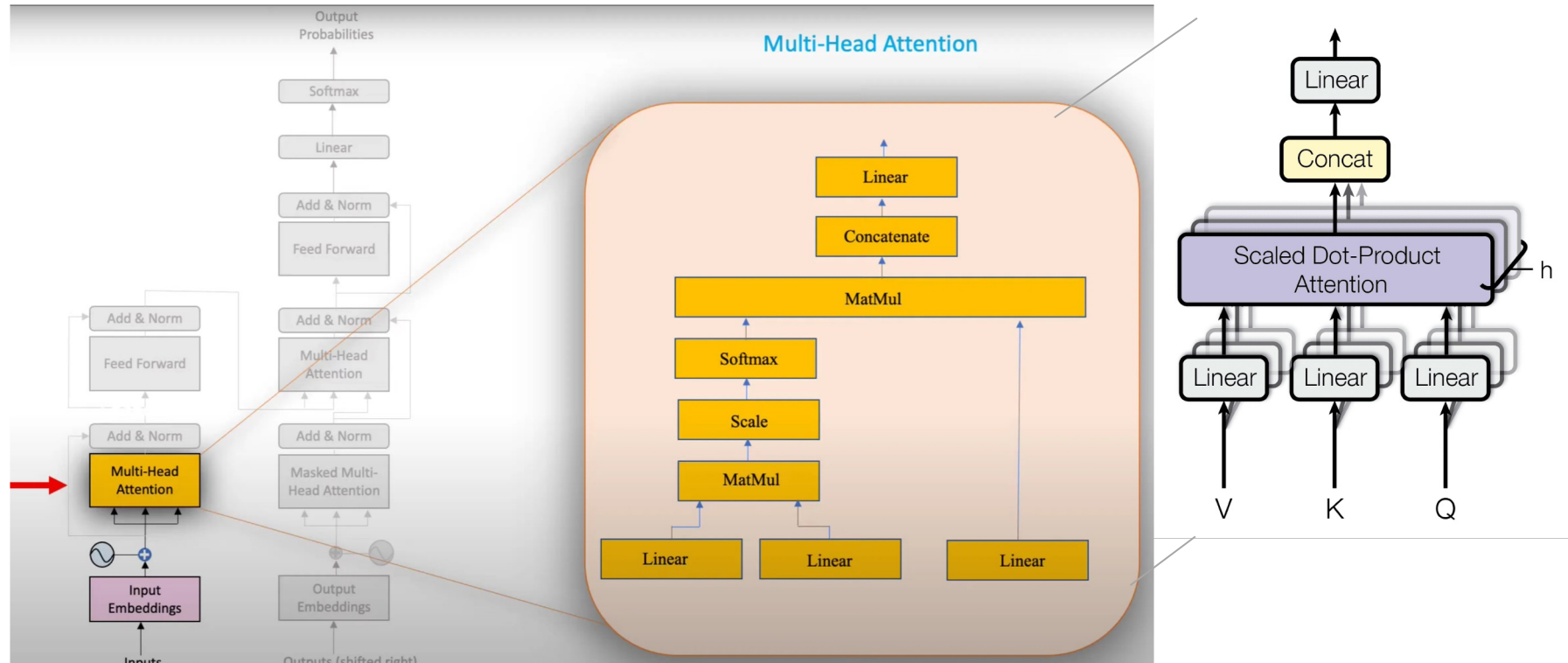


Figure 1: The Transformer - model architecture.

Mechanizm Samouwagi (Self-Attention)



Sources:

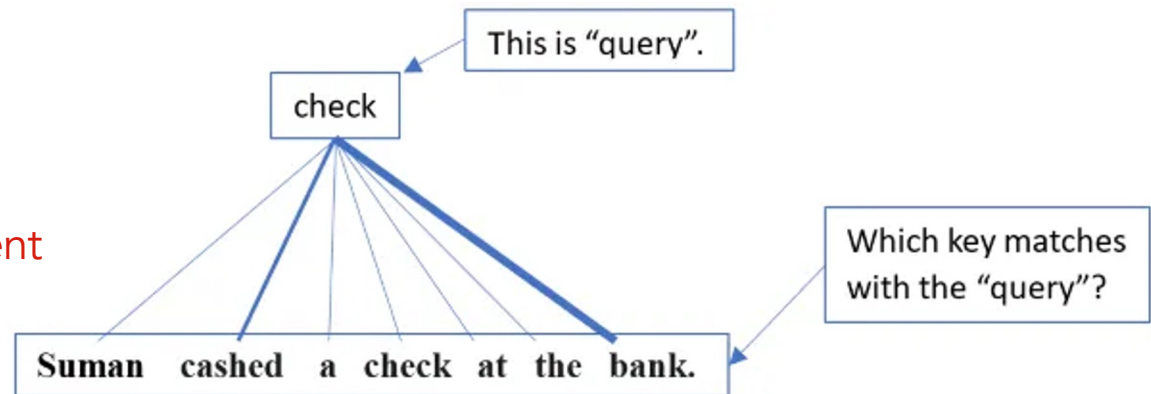
<https://medium.com/@geetkal67/attention-networks-a-simple-way-to-understand-self-attention-f5fb363c736d>

<https://www.youtube.com/watch?v=mMa2PmYJICo>

Self-Attention Mechanism

- Contextual-dependent representation of an element in the sequence

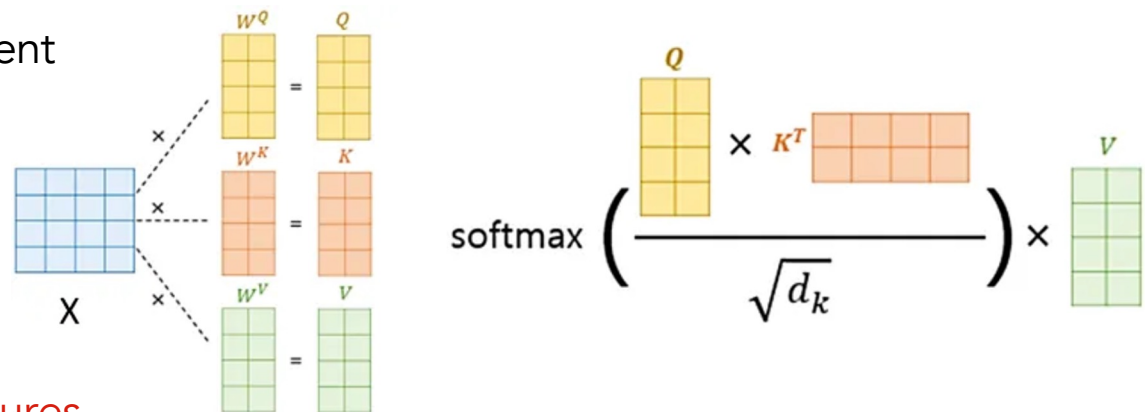
- Inspiration:
Information Retrieval



Source: <https://medium.com/@ramendrakumar/self-attention-d8196b9e9143>

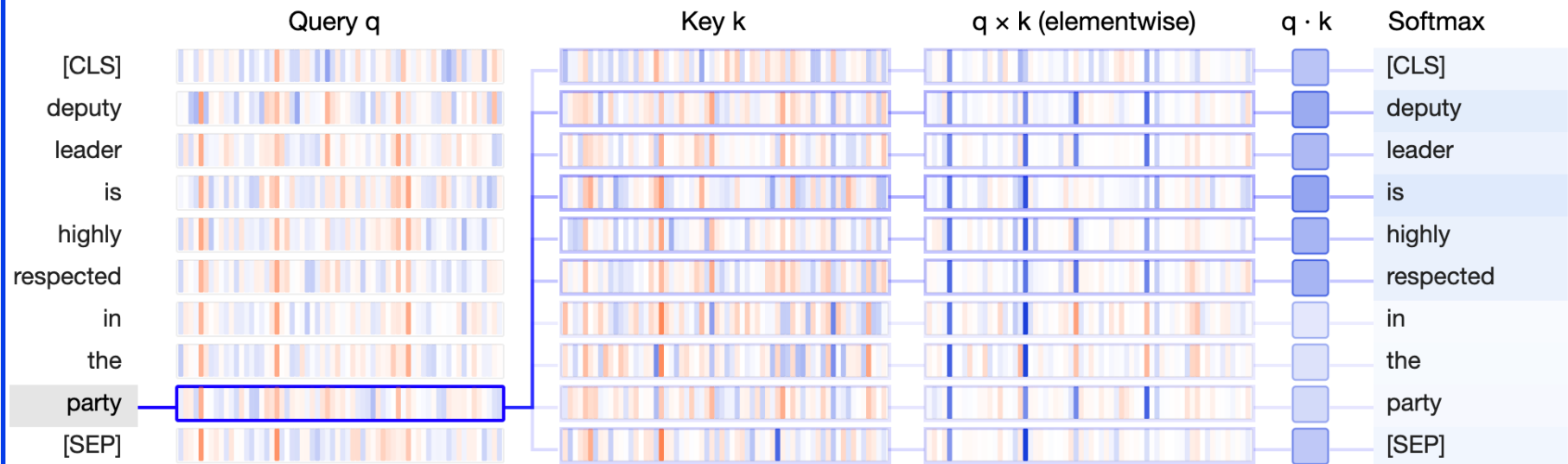
- **Multi-Head Self-Attention:**

- introduces M independent projections Q, K, V
- Goal: to increase the capacity of the network and capture a variety of features (grammar, word relationships, emotions, and other uninterpretable features)



Source: <https://medium.com/@ramendrakumar/self-attention-d8196b9e9143>

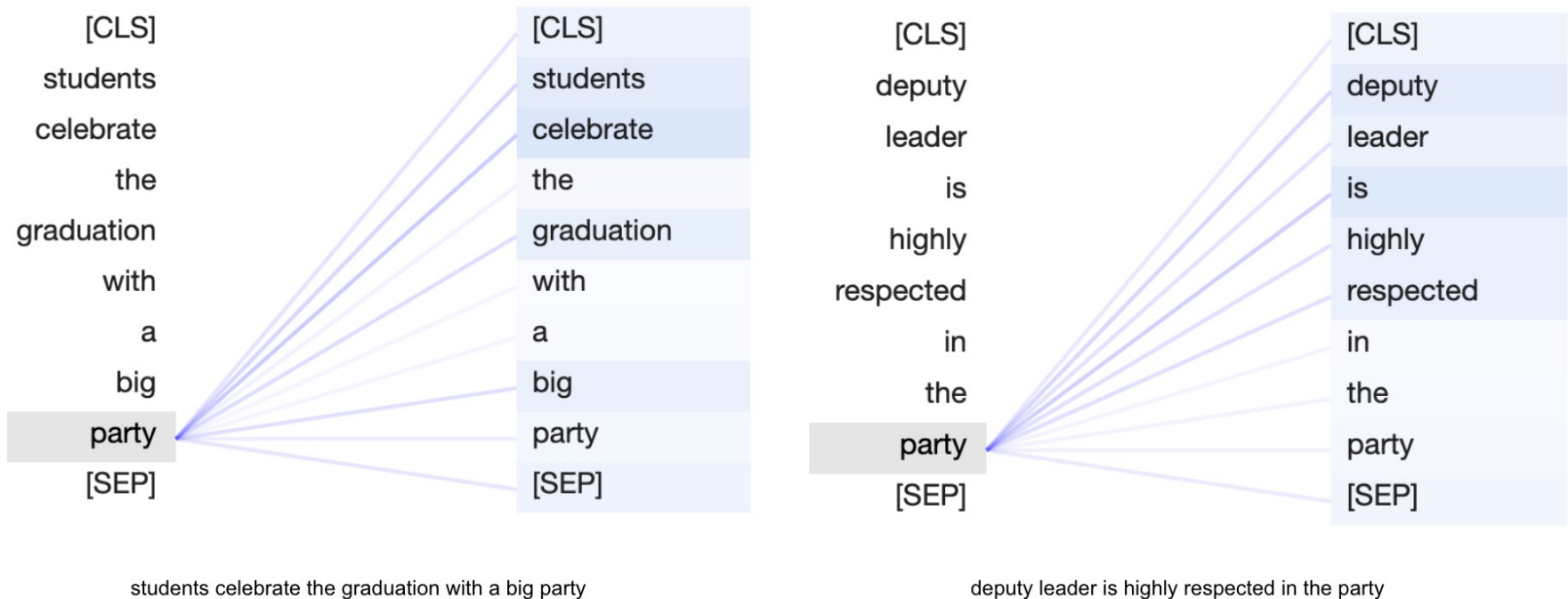
Self-attention – Example (GPT decoder-only model)



Fatih Demirci, Inside GPT – I : Understanding the text generation

<https://towardsdatascience.com/inside-gpt-i-1e8840ca8093/>

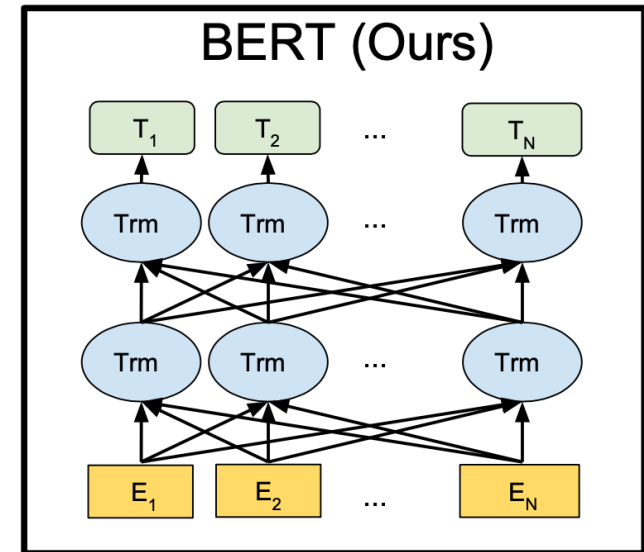
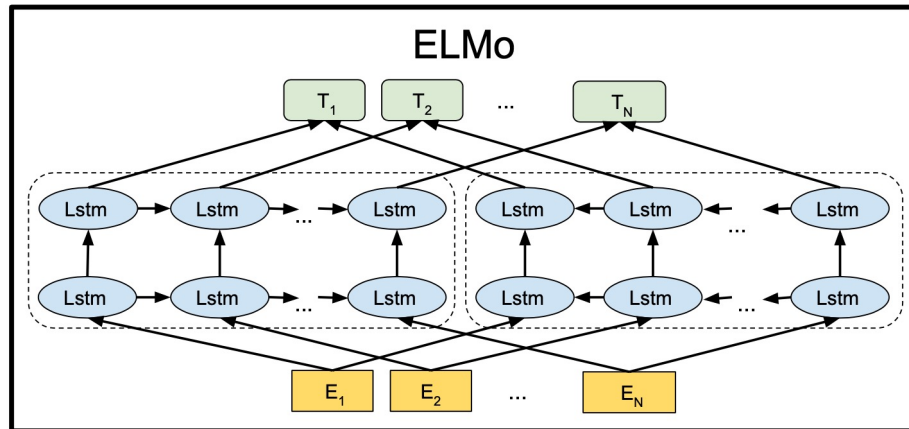
Self-attention – Example (GPT decoder-only model)



Fatih Demirci, Inside GPT – I : Understanding the text generation

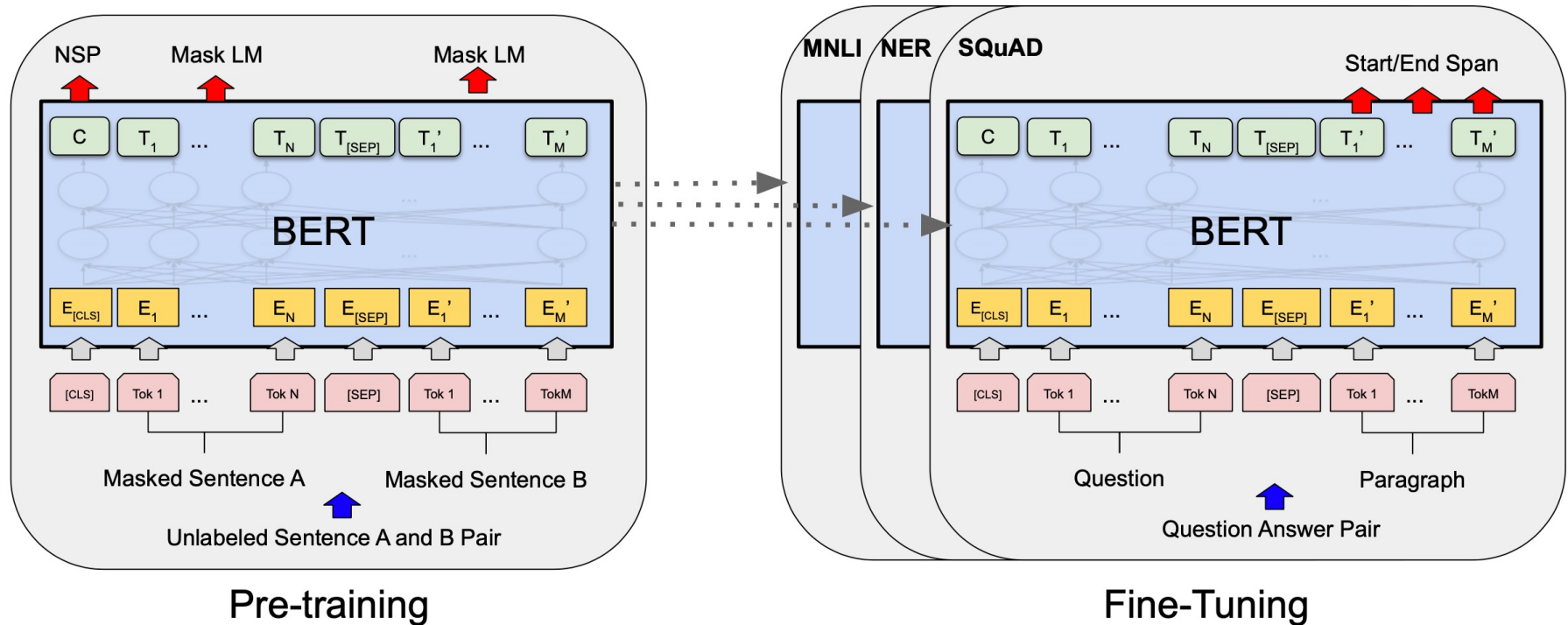
<https://towardsdatascience.com/inside-gpt-i-1e8840ca8093/>

BERT: architecture (Devlin et al., 2018)



(Devlin et al., 2018) CC

BERT: architecture (Devlin et al., 2018)



(Devlin et al., 2018) CC

BERT: Training (Devlin et al., 2018)

- Example of entry:

Input

[CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK] milk [SEP]

Label

IsNext

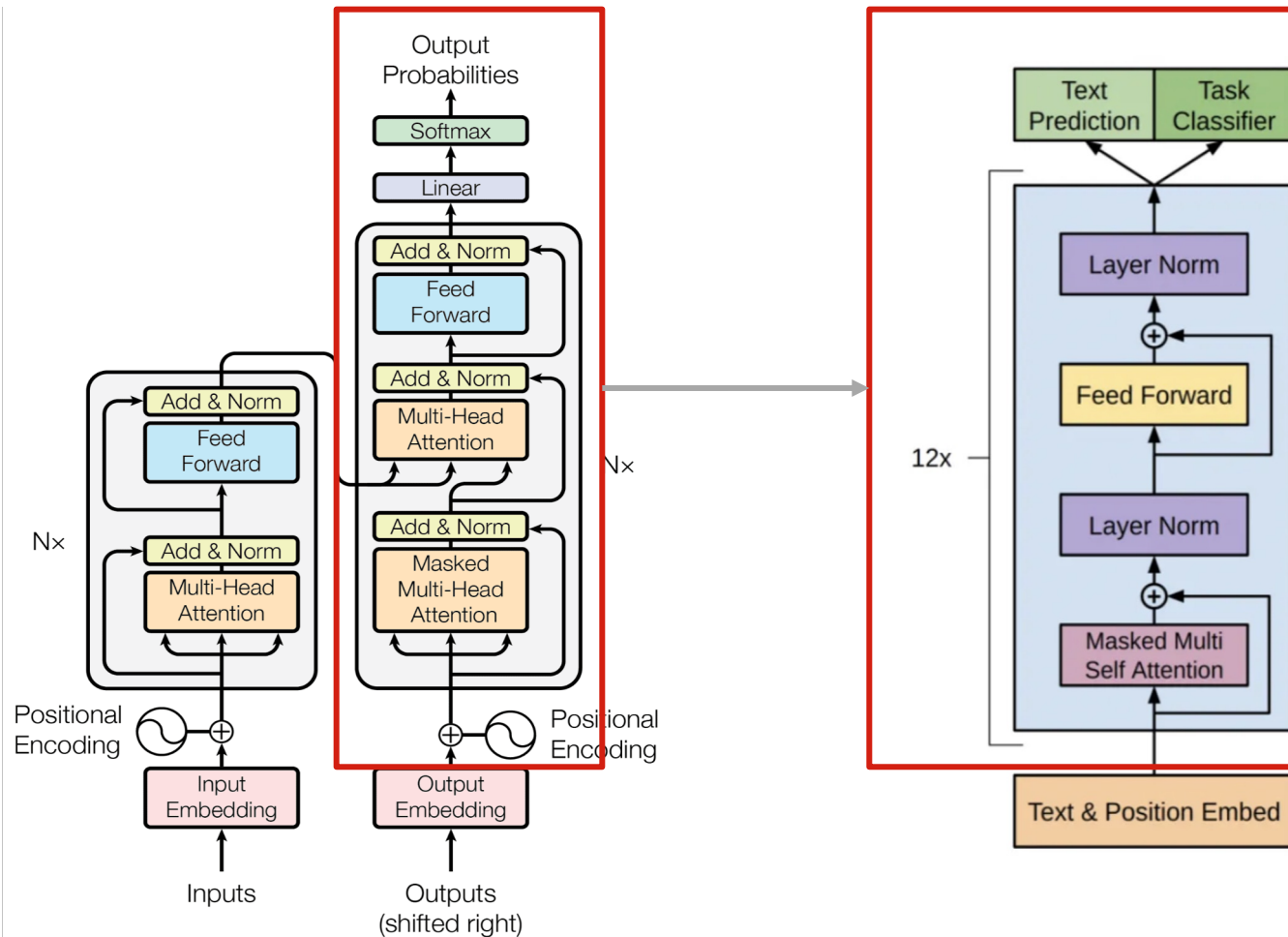
Input

[CLS] the man [MASK] to the store [SEP] penguin [MASK] are flightless birds [SEP]

Label

NotNext

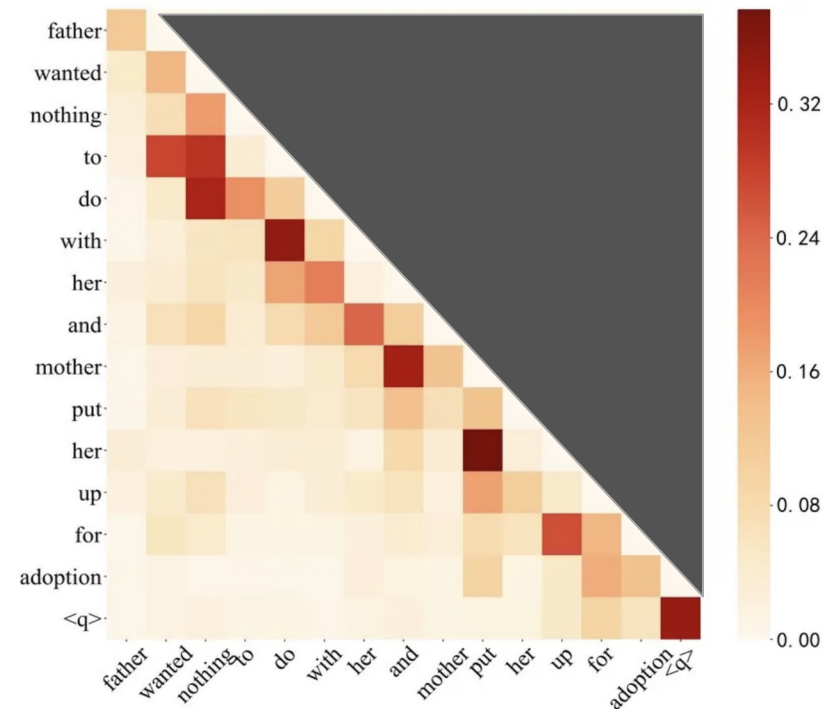
Generative Pre-Trained Transformer (2018)



Generative Pre-Trained Transformer (2018)

Features:

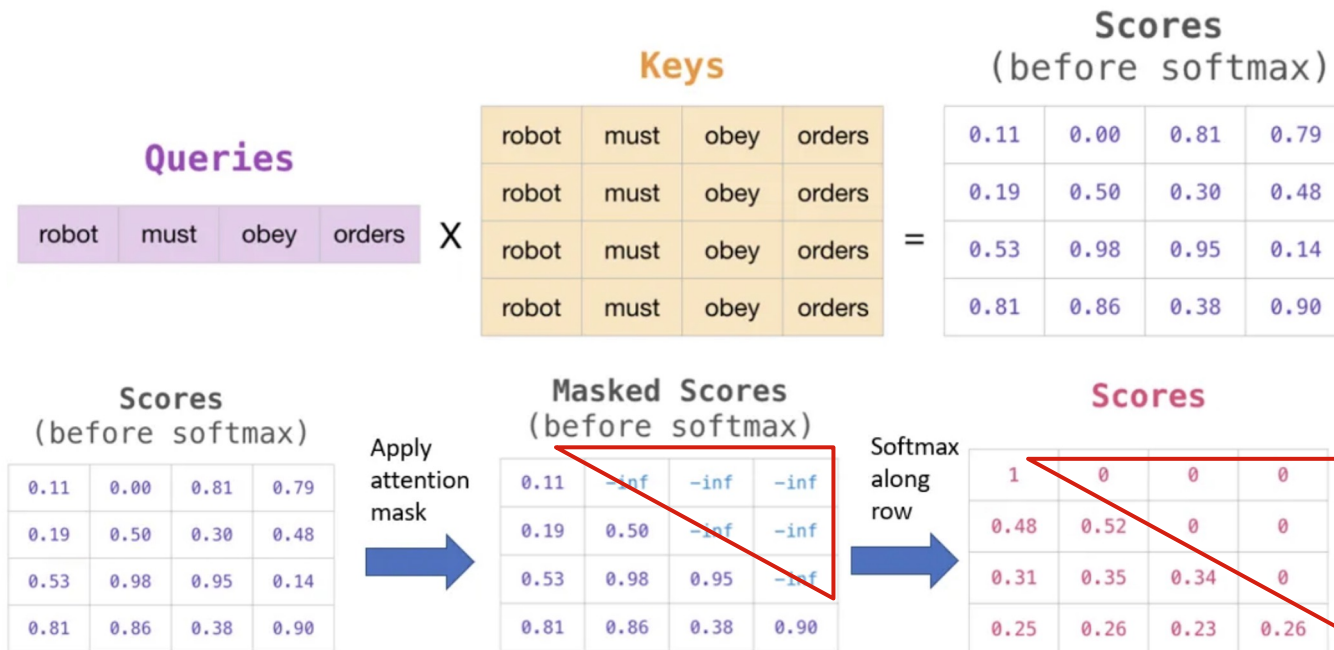
- uses trained positional vectors (Vasvani's transformer used sinusoidal vectors)
- uses a triangular mask so as not to peek at subsequent tokens (**Causal Masking**)



Example output of Query and K^T which is of dimensions Seq_Length x Seq_Length. Causal masking is applied so future tokens are zeroed

Source: <https://medium.com/@AIExplainedML/how-does-the-attention-mechanism-in-gpt-models-work-5f489a59346b>

Generative Pre-Trained Transformer (2018)



Features:

- uses trained positional vectors (Vasvani's transformer used sinusoidal vectors)
- uses a triangular mask so as not to peek at subsequent tokens
(Causal Masking)

Source: <https://medium.com/@YanAIx/step-by-step-into-gpt-70bc4a5d8714>

Generative Pre-Trained Transformer (2018)

- GPT inference:

Step $t-1$: one token x generation at time

Step t : prompt from Step $t-1$ appended with $x \rightarrow$ the model input

Romeo and Juliet is a tragedy written

Romeo and Juliet is a tragedy written by

Romeo and Juliet is a tragedy written by William

Romeo and Juliet is a tragedy written by William Shakespeare

Romeo and Juliet is a tragedy written by William Shakespeare early

Romeo and Juliet is a tragedy written by William Shakespeare early in

Romeo and Juliet is a tragedy written by William Shakespeare early in his

Romeo and Juliet is a tragedy written by William Shakespeare early in his career

Romeo and Juliet is a tragedy written by William Shakespeare early in his career about

Source: <https://towardsdatascience.com/gpt-intuitively-and-exhaustively-explained-c70c38e87491>

Autoregresywne dekodowanie

Strategie dekodowania

- **zachłanna**: wybieranie tokenów o najwyższej wartości softmax
- **próbkowanie**: losowanie tokenów zgodnie z rozkładem softmax (strategie **Top-k**, **Top-p**)
- **wyszukiwanie z promieniem** (ang. beam search)

Dekodowanie zachłanne

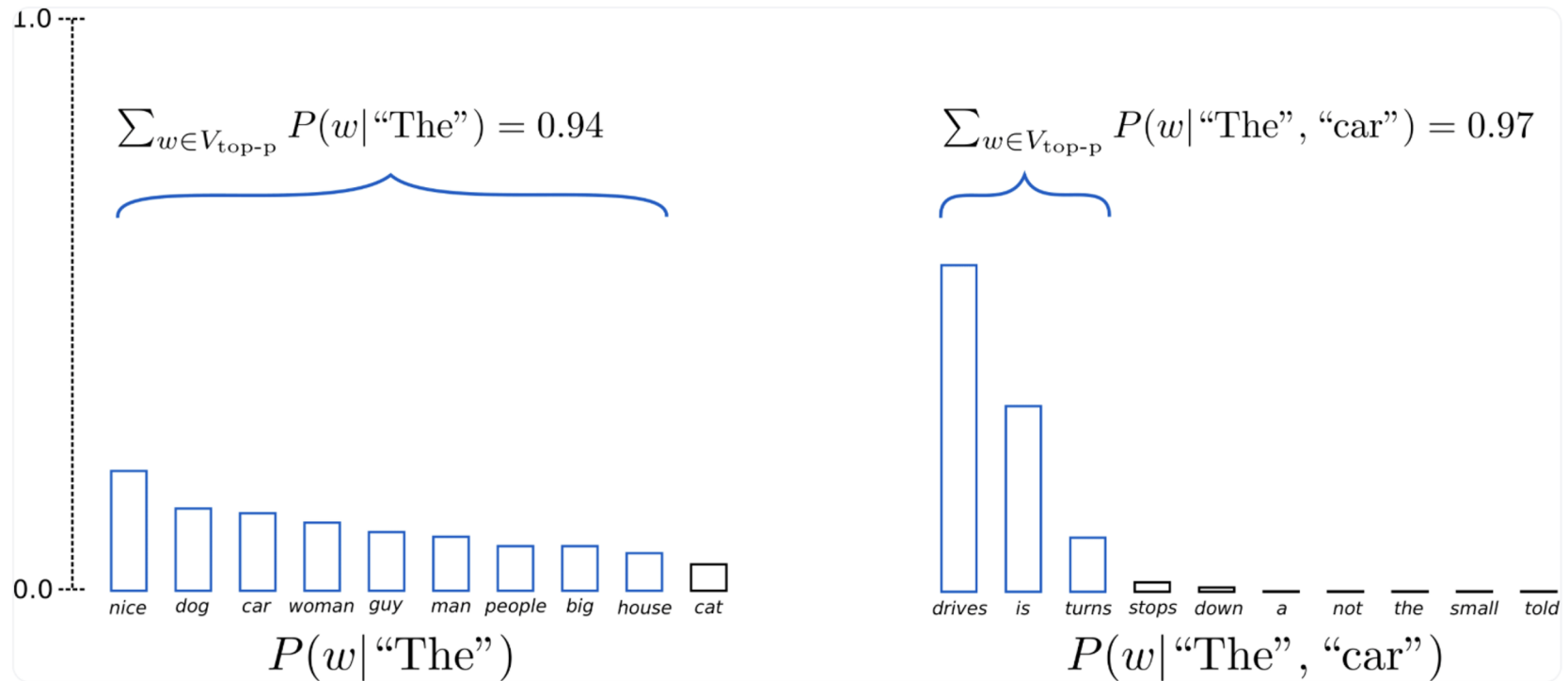
$$y_t = \operatorname{argmax}_{y \in V} P(y | y_1, y_2, \dots, y_{t-1}, x)$$

Źródło: <https://towardsdatascience.com/gpt-intuitively-and-exhaustively-explained-c70c38e87491>

Dekodowanie z próbkowaniem

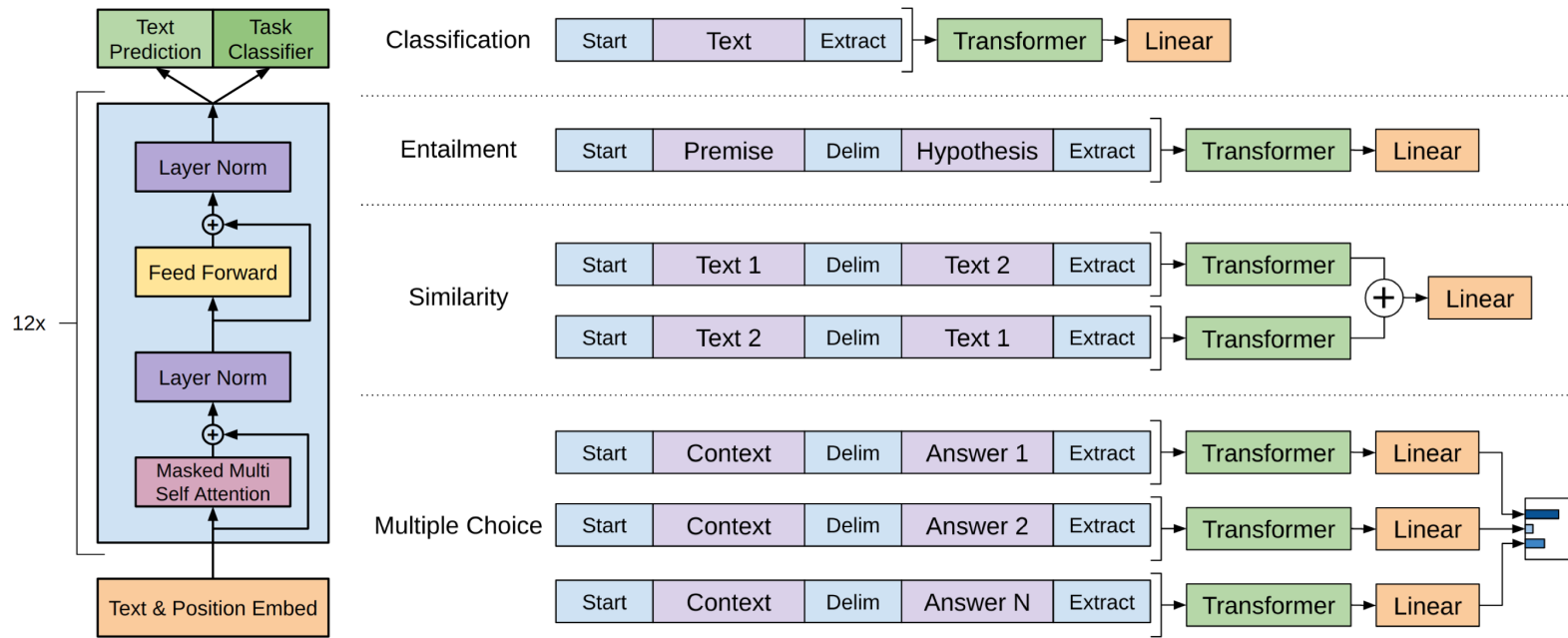
Recommended source for generation (decoding) methods

- <https://huggingface.co/blog/how-to-generate>



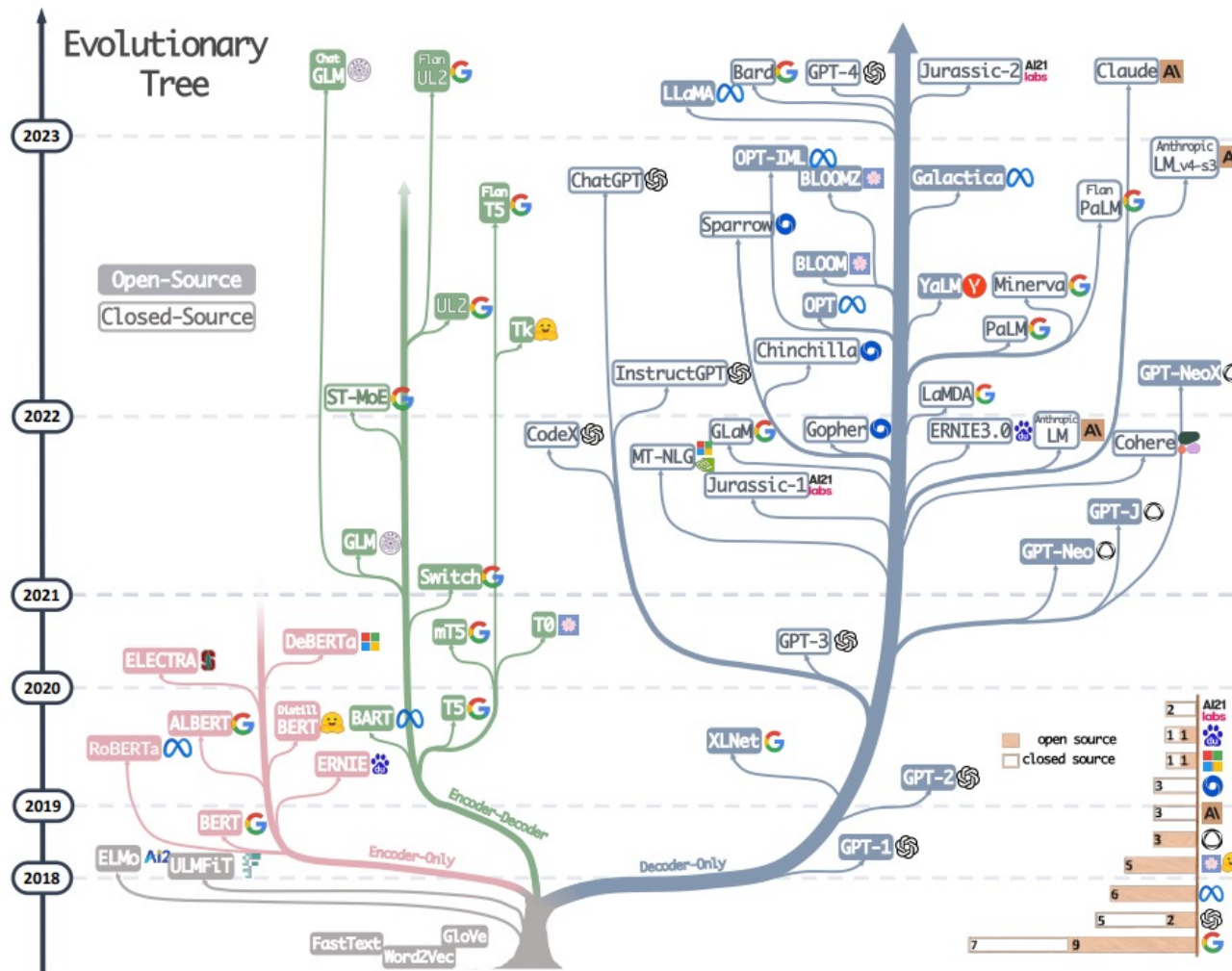
Strategia Top-p

OpenAI GPT model



- Uni-directional decoder following the Transformer model (Radford et al., 2018)
 - <https://www.cs.ubc.ca/~amuham01/LING530/papers/radford2018improving.pdf>

The evolutionary tree of modern LLMs



Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond, J. Yang, 2023

Libraries and frameworks

- Hugging Face (<https://huggingface.co/>) – large scale models repository, provides Transformers library and other tools for deep learning models



- PyTorch is an optimized tensor library for deep learning using GPUs and CPUs.



- PyTorch Lightning - is the deep learning framework build on top of PyTorch, provides appropriate code structure for research and development



- TensorFlow - open-source software library focused on training and inference of deep neural networks.



Literature

- Jurafsky, D. & Martin, J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics and Speech Recognition Prentice Hall, 2000; 3 edition (draft 2023): <https://web.stanford.edu/~jurafsky/slp3/>
- Miroslav Kubat. (2017) An Introduction to Machine Learning. Springer. (available via WUST network)
- Sandro Skansi. Introduction to Deep Learning. From Logical Calculus to Artificial Intelligence. Springer, 2018. (available via WUST network)
- Stuart Russell and Peter Norvig, Artificial Intelligence A Modern Approach, Forth Edition, 2022.
- Charu C. Aggarwal, Neural Networks and Deep Learning A Textbook, 2018
- Ian Goodfellow Yoshua Bengio Aaron Courville, Deep Learning, 2016, <https://www.deeplearningbook.org/>
- JUSTIN SEONYONG LEE Transformers: a Primer <https://www.columbia.edu/~jsl2239/transformers.html>
- Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013a). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013b). Distributed representations of words and phrases and their compositionality. In Advances in Neural Information Processing Systems, pages 3111–3119.